BABEŞ-BOLYAI UNIVERSITÄT CLUJ-NAPOCA

FAKULTÄT FÜR MATHEMATIK UND INFORMATIK

INFORMATIK IN DEUTSCHER SPRACHE

BACHELORARBEIT

TrivAI: Trivia Game mit generativer KI

Betreuer

Lect. Dr. Radu Dragos

Eingereicht von

Cormos Sandra-Andreea

2024

UNIVERSITATEA BABEŞ-BOLYAI CLUJ-NAPOCA

FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ

SPECIALIZAREA INFORMATICA IN GERMANA

LUCRARE DE LICENŢĂ

TrivAI: Joc de trivia cu AI generativ

Conducător ştiinţific

Lect. Dr. Radu Dragos

Absolvent

Cormos Sandra-Andreea

2024

BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA

FACULTY OF MATHEMATICS AND COMPUTER SCIENCE

SPECIALIZATION INFORMATICS IN GERMAN

DIPLOMA THESIS

TrivAI: Trivia Game powered by generative AI

Supervisor

Lect. Dr. Radu Dragos

Author

Cormos Sandra-Andreea

2024

1. Abstract

This thesis explores the development and implementation of TrivAI, a gaming application aimed at providing an engaging and intellectually stimulating experience for users. Beginning with an introduction to the video game industry, the thesis delves into the concept of procedural content generation (PCG) and its significance in game development. It discusses the role of artificial intelligence (AI) in gaming, including its history, applications, and specific use cases in-game AI.

A significant focus of the thesis is on OpenAI's GPT (Generative Pre-trained Transformer) model, highlighting its importance in content generation for games. The thesis identifies and addresses key challenges in game development, such as replayability, maintenance and development costs, platform-specific development, and cross-device progress retention. It proposes solutions to these challenges, leveraging technologies like Unity game development engine and PlayFab backend services.

The implementation process of "TrivAI" is detailed, covering the integration of PlayFab and OpenAI, the use of package managers, and the design and logic of the application. The thesis also explores the visual and interactive design of the application, including user interface components, and explains the application's logic and implementation, covering user authentication, game initialization, question generation, user interactions, scoring, leveling system, and leaderboard display.

Future improvements and potential enhancements for the application are discussed, providing insights into possible directions for further development. The thesis concludes by summarizing key findings and reflections on the project's achievements, underscoring the significance of innovation and technology in creating immersive gaming experiences with educational benefits.

Table of Contents

2. Introduction

The result of this project is meant to represent my abilities and skills to develop a full game from scratch, including designing, drafting, implementing, and testing – with each stage to be handled with the best practices considered on the current market.
TrivAi is a trivia-type game designed to offer players greater control over the game, enhancing both replayability and immersion. By allowing players to select desired categories of trivia questions and using artificial intelligence to generate questions in real time, TrivAi provides a dynamic and personalized game experience that evolves with each game. Moreover, it utilizes OpenAI GPT's memory capacity so that there is a minimal likelihood that questions will be repeated. This way every player can enjoy a different gameplay every time engaging in a new, fresh experience.

Through these innovative features, TrivAi aims to redefine the traditional trivia game genre, providing endless entertainment and engagement for players of all backgrounds.

During the preparation of this work the author used OpenAI GPT in order to enhance clarity and assist with writing. After using this tool/service, the author reviewed and edited the content as needed and takes full responsibility for the content of the thesis.

2.1. Introduction to the video game industry:

In recent decades, the video game industry has experienced exponential growth, with games coming to be seen as much more than an escape from the daily grind, a recreational break, but a cultural phenomenon and a major sector of the entertainment industry. This industry has become one of the most successful, largest, and fastest-growing entertainment industries globally. It generates billions of dollars annually and it doesn't seem to stop here.

As illustrated by the graph in Figure 1. the profitability of games is increasing significantly, as can be seen, more than tripling in the last seven years, with expectations of reaching much higher amounts in the coming years. This trajectory is inevitable given the constant development of technologies and the integration of artificial intelligence (AI) into game development processes.

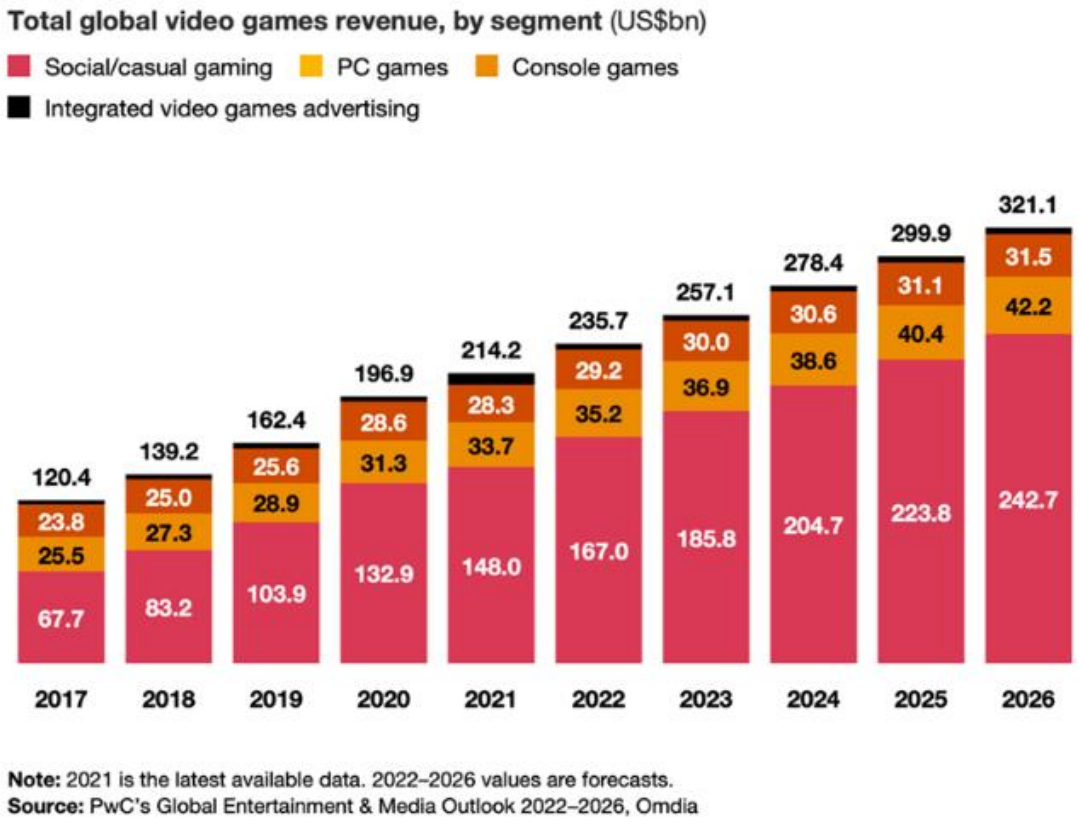


Figure 1. Total Global Games Revenue by Segment

As the industry expands, so too does its audience. While traditionally associated with younger males, the demographics of video game enthusiasts have significantly diversified in recent years. Female gamers now constitute a substantial portion of the gaming community, as can be seen in Figure 2 alongside individuals spanning all age groups.

The popularity of phone games and simple little games with few rules has managed to broaden the appeal of video games to older groups of people. The mobile gaming industry is booming, with global revenue expected to reach more than $200 billion in 2025 as shown in Figure 3.
Hyper-casual games were the main driver of downloads. On a global level mobile games accounted for more than 21% of all active apps on the App Store in 2021, according to gitnux.org/mobile-games-statistics.

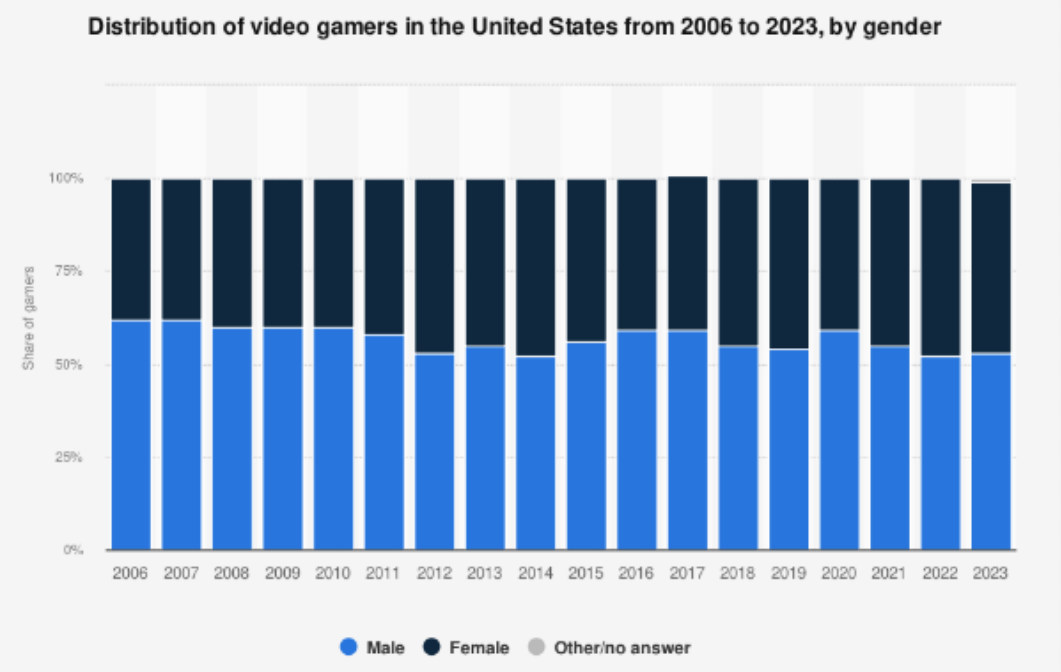


Figure 2. Distribution of video gamers in the USA
(Source: Entertainment Software Association, July 2023)

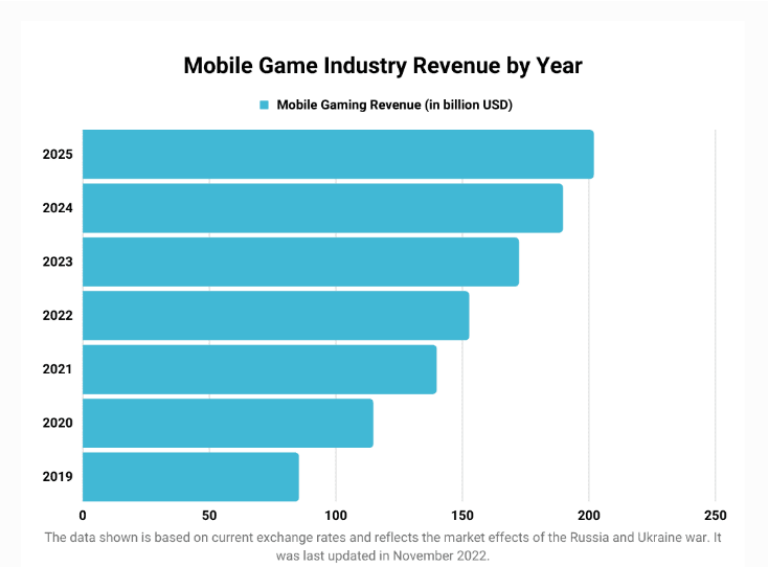


Figure 3. Mobile Game Industry Revenue by Year

(Source: Entertainment Software Association, April 2023)

Because of the new technologies that are emerging and have emerged lately, the future of video games looks increasingly promising. The freedom to create something new and creative is in the hands of anyone who has the desire to bring something new to the market.

Games have progressively become more complex, leveraging graphics, player interactions, storytelling, challenges for players to overcome, and intricate game mechanics. These elements are meticulously crafted to imbue players with a sense of competitiveness, excitement, thrill, achievement, satisfaction, immersion, and escapism. Titles like World of Warcraft (WOW), The Sims, Call of Duty, and Hogwarts Legacy, alongside smaller mobile games like Angry Birds, Hay Day, and Candy Crush, have captivated billions of people, boasting thriving communities with thousands of active players.

Certain games are incredibly captivating, with players dedicating 61-80% of their time, especially in cooperative or team matches.

However, for a game to remain relevant, not to go out of date too quickly, and to be a lasting success it must be engaging enough for the group of people it is focused on. This can be achieved by Procedural Content Generation (PCG).

2.2. Introduction to Procedural Content Generation

Procedural content generation (PCG) serves many uses in game design. Perhaps its most common use is to enhance replayability, as diverse content can lead to wildly different gameplay experiences. It has also been used to bypass technical limitations. to build games that adapt to the skill level or preferences of the user, either during runtime or offline.

One of the first examples of PCG is the game “Rogue”, made in 1980.
“Rogue” is a dungeon-crawling game created in the 1980s. Players explore the levels of a dungeon to find Yendor's Amulet. They encounter monsters and collect treasures such as weapons and potions. The game is turn-based and uses ASCII graphics. The game's levels and items are procedurally generated, providing a unique experience for each game, and improving replayability using PCG.

A linked concept to replayability is adaptability: the use of procedural content generation to adjust content according to the player's actions or skill levels. Adaptability is a way to improve the replayability of a game, and also a possible way to diversify the player base by encouraging players with different play styles or skills.

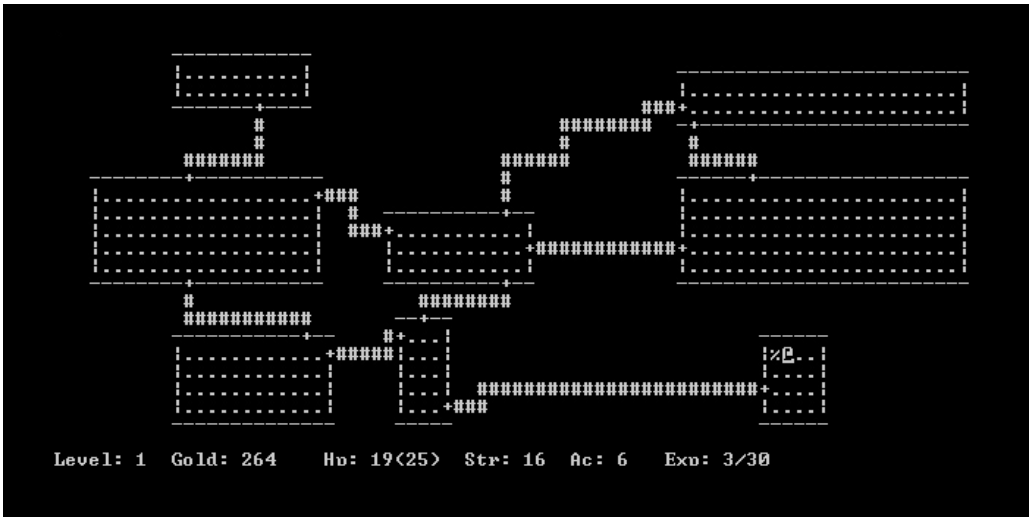


Figure 1: Example of Procedural Content Generation in Games
(Source: Rogue, Developed by Michael Toy and Glenn Wichman, 1980)

2.2.1 Player Control Over Content

Many games that incorporate PCG tend not to let the player's eyes glaze over the generator. For those games that do provide the player with access to the generator, there are two forms of access offered: indirect and direct.

Indirect control is often used in games that adapt in real-time to the skill level of the player, such as the “Polymorph project”. A player could choose to do certain actions, such as playing more unskilled or being killed by enemies, which would result in the level adapting to become easier, thus reducing the occurrence of certain difficult combinations of level components

On the other hand, direct control gives players direct access to this generator, giving them the ability to change the course of the game as they please. In “The Sims”, players have the power to sculpt every aspect of their Sims' appearance and personality. From choosing hairstyles and outfits to defining traits and aspirations, the personalization options are vast. This level of control allows players to create Sims that mirror themselves, their friends, or even their favorite fictional characters. Whether they want the possibilities are endless. In addition, players can experiment with different combinations of features and appearances to create diverse and dynamic Sims that bring their virtual neighborhoods to life. Through this direct access to procedural content generation, players become the architects of their virtual worlds, encouraging creativity and personal expression in the gaming experience.

Also, there are games like “Minecraft” where although there is no direct access to this generator, the player can customize the speed at which they can explore the procedurally generated content, as well as the size of the space, type of biomes, etc. but they have no control over what they find there. Each world starts from a seed: which is a number used to initialize the terrain generation and everything it will contain. It means that given the same initial conditions - the same seed - they will always produce the same results. And therefore, every Minecraft world can only be recreated from its seed. And since the seeds themselves are stored using 64 bits, there are 18.4 quintillion unique values and unique worlds, that can be generated.

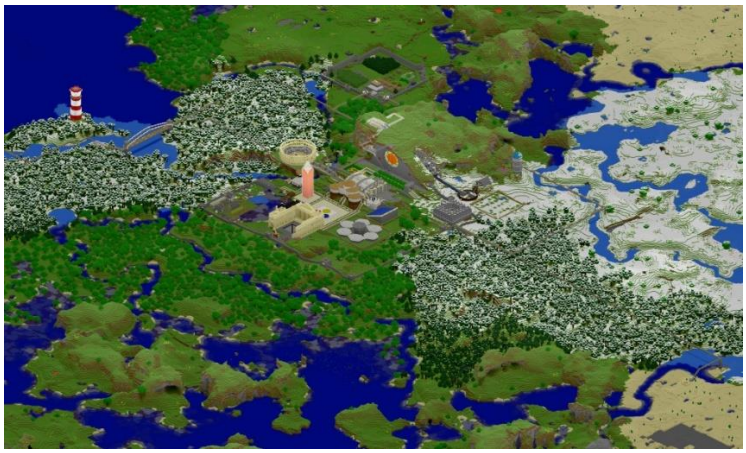


Figure 2: Example of Indirect Procedural Content Generation in Minecraft
(Source: Minecraft, developed by Mojang Studios, 2009)

2.2.2 Techniques Used In PCG

PCG techniques consist of a multitude of algorithms for generating dynamic content, from characters to environment. With this key strategy, you can have infinite possibilities for entertainment, discovery, and exploration, taking games to a whole new level. The manpower becomes smaller and the benefits to the industry become greater. A game that contains these procedural content generation techniques can increase replayability and provide a unique and unpredictable experience to anyone who tries it.
Here are some examples of techniques and frameworks commonly used in PCG:

- Random generation algorithms, which generate content based on random numbers and seeds.
- Rule-based generation, where predefined rules or algorithms determine how content is generated based on specific criteria.
- Fractal generation, which uses mathematical algorithms to create complex and natural-looking patterns and landscapes.
- Cellular automata, a type of computational model, used in PCG to simulate dynamic and evolving environments, such as terrain or vegetation growth.

In addition, grammar-based approaches provide structured frameworks for content generation:

- L-system uses a set of rules to describe how an initial string is iteratively transformed by applying production rules. These rules define how each symbol in the string is replaced by another set of symbols.

  They are particularly suitable for modeling plant and tree growth, as well as for generating natural-looking terrain features and patterns.
- Shape grammars, on the other hand, are good at modeling architectural or artistic compositions. They can be applied in procedural content generation to generate diverse and aesthetically pleasing environments, such as cities with varied architectural styles or landscapes with distinct geological formations.

2.3. Introduction to Artificial Intelligence

In today's ever-evolving world, artificial intelligence (AI) is not just a tool but a game-changer, shaping the way we live, think, and play. As emphasized in the following quote: *"Artificial intelligence (AI) systems already greatly impact our lives — they increasingly shape what we see, believe, and do. Based on the steady advances in AI technology and the significant recent increases in investment, we should expect AI technology to become even more powerful and impactful in the following years and decades"* [1]

In this chapter of the thesis, we will explore the great power of artificial intelligence, with a particular focus on its application to games. Games, once restricted to entertainment, have appeared as fertile ground for artificial intelligence innovation, leading to developments beyond mere recreation. By analyzing the intersection between artificial intelligence and games, we gain valuable insights into the capabilities and potential of AI technology.

While this exploration focuses primarily on the field of games, it is essential to briefly address the broader AI landscape. For a better understanding of the integration and impact of AI, the fundamentals and developments in recent years will be briefly explained in the next chapters.

2.3.1. AI, Machine Learning, and Deep Learning: Differences Explained

Firstly, the main differences between AI, Machine Learning and Deep Learning:

- AI is a broad concept that refers to the techniques that a machine uses to mimic the intelligence of a human by being able to learn, improve over time, and have logic in making decisions.
- Machine Learning is a subset of artificial intelligence that involves several algorithms that focus on the idea that a system can identify specific patterns, learn from certain data, and make decisions with minimal human intervention.
- Deep Learning is a subset of machine learning that uses multi-layer neural networks to train computers to process data similar to a human brain. Deep learning models can recognize complex patterns in images/pictures, sounds, and text.

The main purpose of Deep Learning is to produce accurate insights and predictions. They are often used to automate tasks that require human intelligence like describing the details of a picture or transcribing a sound into text.

2.3.2. History of Artificial Intelligence

Artificial Intelligence has been through a lot of development and research to reach the point where it is today. Let's dive into the history of artificial intelligence to better understand its progress.

In **1950**, Alan Turing created the Turing Test. This test can determine if a machine demonstrates human intelligence. The Turing Test consists of a judge, a person, and a machine. The judge is addressing some questions to both of them and he has to evaluate the answer and declare which one is which. If the judge can't tell which one the human is, the computer has succeeded in demonstrating human intelligence. That is, it can think.[3]

Then in **1956,** the Dartmouth Conference took place. Many experts from different fields were brought together to discuss the creation of intelligent machines. This conference marked the start of formal AI research.

In the same year, the first AI program appeared. *Logic Theorist,* invented by Allen Newell, Herbert A. Simon, and Cliff Shaw, is a program focused on problem-solving and also theorem proving in the mathematical domain. It had a great impact because of its ability to mimic human-like reasoning, and to generate proofs of mathematical theorems.

Over time science people were more and more curious about the power that artificial intelligence possesses. In **1962** Arthur Samuel, a pioneer in the field of artificial intelligence dedicated all his work to the pursuit of creating an intelligent machine. His quote, *"I became so intrigued with this general problem of writing a program that would appear to exhibit intelligence that it was to occupy my thoughts during almost every free moment for the entire duration of my employment by IBM and indeed for some years beyond,*"[4] encapsulates his passion and desire to the

advancement of AI. Samuel developed the first self-learning program, more precisely a checkers-playing program. His work laid the foundations for today's machine-learning techniques.

Three years later Joseph Weizenbaum created ELIZA, the first chatbox that uses artificial intelligence to respond.

After that in the following years, the research fund for AI decreased, leading to an "AI Winter", which is a period during which there is reduced interest and funding for major AI research.
Because of technological advances such as faster processors and improved algorithms, practical applications like medical diagnosis, robotics for automation, strategic investments, and academic interest the “AI Winter” started to slowly fade away.

Thus, projects like IBM's Deep Blue started to catch the eye of scientists. Deep Blue defeated 1997 chess world champion Garry Kasparov in a six-game match.
Then in 2011, IBM's Watson defeated human champions in *Jeopardy!,* a quiz competition. The idea of sending a machine learning system to compete on "Jeopardy!" originated with David Ferrucci, an AI expert then at IBM. Ferrucci's interest was making machines that "*actually understood language in a much deeper way”*. The main obstacle was that Watson would need to be able to parse the language of Jeopardy questions.

Curiosity and enthusiasm about AI increased drastically since its low point in the early 1990s. As of the year 2012, all that concerns artificial intelligence grew leading to the current (as of 2024) AI Boom.

Now we are surrounded by intelligent machines and computers that help us in everyday life, from self-driving cars to robots that are used to identify diseases, to Chat GPT, one of the largest and most powerful language models up to date, that everyone has heard about at least once.

2.3.3 Applications of Artificial Intelligence

With the passing of time, Artificial Intelligence programs are becoming increasingly common in a large variety of fields. They are used to improve efficiency, productivity, and sustainability.

Among the many uses of AI, it can be used to collect data from different sources, analyze data and identify patterns, create data visualization for a better understanding of data, and make recommendations and insights.

- Healthcare: By analyzing patient data, AI can identify diseases, help doctors diagnose diseases earlier and accurately, can help develop new drugs and treatments tailored to the patient's specific needs.
- Education: AI can be used to create personalized learning programs and courses, keep track of students' progress, and free up teachers' time so they can focus more on teaching itself.
- Agriculture: Programs that analyze soil conditions, and weather patterns are a very big help to farmers. They can easily reduce costs by automating tasks and monitoring their resources, water and soil.
- Manufacturing: Automating repetitive tasks not only reduces fabrication costs but also decreases the margin of error and improves quality
- Finance: In today's world cryptocurrency and investments have captured the interest of more than a quarter of the population Therefore, it could not miss AI programs that help making investment decisions and manage risks
- Transportation: AI is being used to develop self-driving cars and improve traffic management
- Etc.

These are only some of the applications of AI and there are so many more to come and be further developed.

The diagram in Figure 4 illustrates the significant growth of the Artificial Intelligence market size from 2021 to 2030. The diagram consists of columns representing the billions of dollars for each

year. It shows how the market size has constantly evolved over the years showcasing the continued expansion of the AI market into the future. Based on the last years the market size increased by close to 200% from 2021 to 2024 and considering the steadily advancing technology and the impact AI had so far it is highly likely that the integration of artificial intelligence into various aspects of our lives will continue to deepen and expand in the coming years even more.

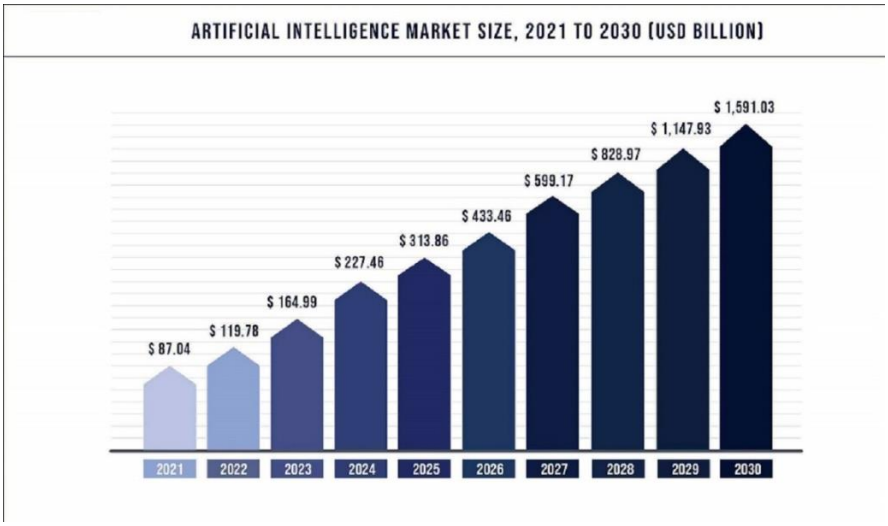


Figure 4: Artificial Intelligence Market Size (2021-2030)

Source: (Technosys.com, published by TARUN NAGAR, Feb 2023)

2.3.4. Game AI

In the broadest sense, most games incorporate some form of artificial intelligence (AI). However, due to the diverse range of game genres and characters, defining what constitutes Game AI requires a broad interpretation.

The primary goal of Game AI is to create an immersive experience that maximizes player enjoyment and fosters a sense of empowerment. For instance, developers have utilized AI for years, from classic arcade games like 'Pac-Man,' where AI controls ghost movements to create dynamic and challenging gameplay, to modern titles such as 'The Sims,' 'Skyrim,' and 'The Witcher 3.'

Artificial Intelligence in games serves various purposes, as outlined in the academic article 'AI for game production' [6], which categorizes Game AI into three approaches: AI as Actor, AI as

Designer, and AI as Producer. To better understand these classifications, examples are provided to illustrate how AI is utilized in each category:

AI as Actor

- More intellective non-playing characters (NPCs) and agents. Because most interactions players come across in games are NPC's, they were enhanced with AI. They become more intelligent and responsive, shaping the attitude of the player and maintaining the rhythm of the conversation. These enhancements have elevated the quality of player-NPC interactions giving a much greater sense of reality

AI as Designer

- AI algorithms can be employed to handle collision detection in games, ensuring that objects interact with each other realistically and preventing objects from passing through one another. This is crucial for creating immersive and believable game environments.
- Balancing the difficulty level of a game according to the user's experience is crucial. It's essential to match players with challenges of a moderate level to prevent frustration from overly complex tasks. This principle applies to both single-player and multiplayer (PVP) games, where players of similar skill and proficiency levels should be matched.
- Creating game content: Creating texts, music, and images by AI is a great technique used often in game development for their convenience and efficiency to reduce the cost of designers and also allows for the creation of dynamic and varied game experiences that adapt to the player's actions.
- Pathfinding algorithms allow characters and entities within a game to navigate through complex environments efficiently. Whether it's a player-controlled character or a non-playing character (NPC), AI-driven pathfinding ensures smooth and realistic movement within the game world.

AI as Producer

- Dynamic content generation: AI-based procedural content generation techniques can be used to create dynamic and varied game content tailored to individual player preferences. For example, artificial intelligence algorithms can generate customized levels, quests, or

challenges based on skill level, play style, and player progression. Games such as "*Minecraft"* and "*No Man's Sky"* use procedural generation algorithms to create large and diverse game worlds that adapt to player actions and preferences.

- Content recommendation systems: AI-based recommendation systems can suggest customized game content, such as levels, missions, or in-game purchases, based on player preferences and behavior. Games such *as "AI Dungeon"* use AI-generated content to create interactive storytelling experiences.

The Game AI is nowadays in most games incorporated in some kind of form. Whether through smarter NPCs, better collision handling, or creating game content, many companies try to adapt to these advancements to enhance player experiences and stay competitive in the ever-evolving gaming industry. *"Game AI is charged with taking progressively more responsibility for the quality of the human player's experience in the game"*[5]. Therefore AI in the gaming market had a progressively big growth as can be seen in Figure 5. It can be easily seen that AI, either used for non-player characters or level generation, image enhancement, scenarios, and stories, or balancing the in-game complexity, the incorporation of AI increased and will increase even more.

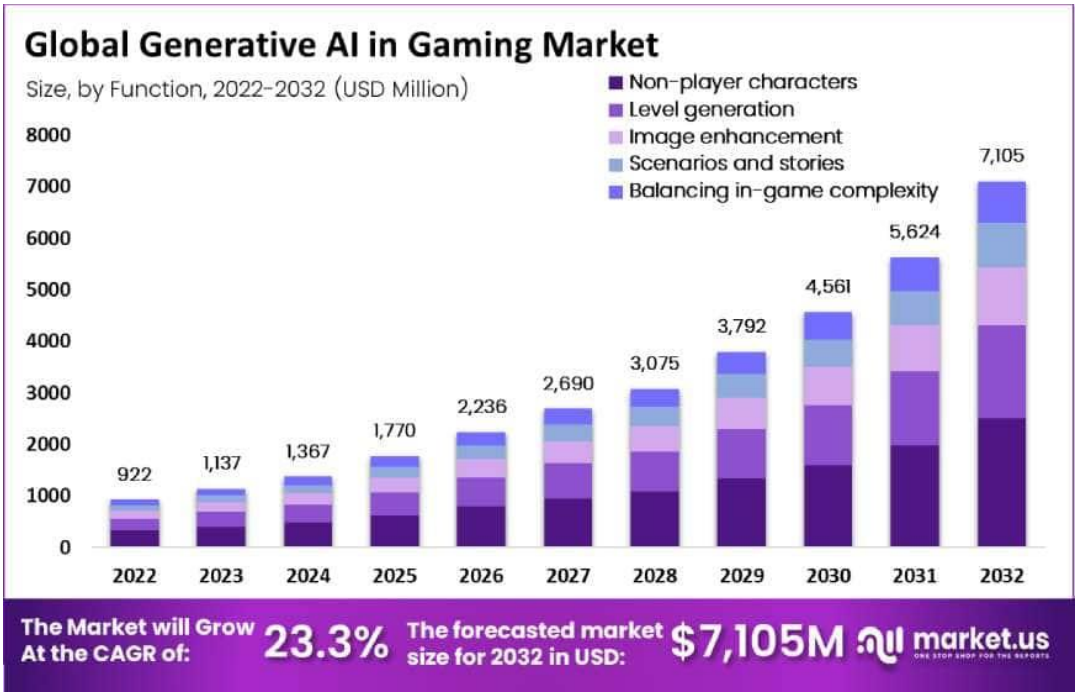


Figure 5: Global Generative AI in Gaming Market

Source: (market.us, March 2024)

3. Problem

One of the biggest challenges in game development is ensuring that a game is engaging and can stand the test of time. At the heart of this challenge is the notion of replayability - the ability of a game to remain engaging and enjoyable after several attempts. Developers strive to create experiences that transcend the initial excitement of discovery, offering depth, variety, and unpredictability to keep players coming back for more.

This chapter delves into the primary issues confronting game developers today, focusing on the high maintenance and development costs, the continuous need for fresh content, and the critical challenge of ensuring replayability.

3.1. Replayability Issues

Replayability is a critical factor in the longevity and success of a game. To keep users engaged in the game, updates need to be constantly released. These updates can include new modes, heroes, features, events, weapons, and more empowering players with meaningful choices and consequences. This can enhance replayability and increase the game's chances of persisting over time.

However, achieving this level of replayability requires substantial effort. Game developers need to maintain existing functionality by fixing bugs and resolving performance issues. As a result, developers are constantly tasked with a substantial volume of work. To manage these tasks successfully, it is essential to create and follow a clear development plan. This plan should outline specific tasks that developers can realistically accomplish in the given timeframe.

3.2. Maintenance costs

Maintenance costs are another significant challenge in game development. Once a game is released, the work is far from over. Developers must continually update and debug the game to fix bugs, patch security vulnerabilities, and ensure compatibility with new hardware and software updates. This process requires a dedicated team of developers and support staff, whose salaries and operational costs can be substantial. In addition, maintaining servers for online multiplayer

games adds another layer of financial burden. These high maintenance costs can strain the resources of a development studio, especially smaller independent developers, and can impact the overall profitability and sustainability of the game.

3.3. Content Development Costs

Creating attractive, high-quality content is another major challenge. Modern players expect a rich and immersive experience, which requires substantial investment in graphics, sound design, narrative development, and game mechanics. The cost of producing all these features to keep the content fresh is considerable. Additionally, because the players become familiar with game content over time, it necessitates continuous updates and additions of new elements to maintain the player's interest and engagement.

3.4. Platform-Specific Development

Another problem is that some games are specifically designed to run only on a certain platform such as Android or iOS. However, the support for both platforms increases costs due to the need for platform-specific optimizations. The selection of a technology stack, including game engines and frameworks, impacts both development time and costs. Cross-platform game development requires additional resources and testing to ensure consistent performance and user experience across devices. This is adding to the complexity and expense of the development process, posing yet another hurdle for developers.

3.5. Cross-device progress retention

Ensuring cross-device progress retention is a major challenge in game development. Players expect to switch between devices—like smartphones, tablets, and PCs—without losing game progress. This requires robust cloud saving and synchronization to handle user authentication, data storage, and real-time updates. Without these, players may face data conflicts, overwrites, or lost progress, leading to frustration and a poor gaming experience.

4. Solution

In the previous chapter, we identified several major challenges in game development, including ensuring replayability, managing high maintenance and development costs, creating continuous fresh content, and supporting multiple platforms. This chapter outlines the solutions implemented in the development of TrivAi to address these challenges effectively.

4.1. Strategies for Enhanced Replayability and Content Generation

Dynamic Question Generation:

To solve the problem of replayability, TrivAI employs OpenAI's GPT to generate trivia questions in real time. This approach ensures that each playthrough offers a unique set of questions, minimizing repetition and maintaining player interest. By using artificial intelligence, the game can continuously provide new content without the need for manual updates.

Player-Defined Categories:

TrivAi enhances player engagement by allowing users to select their trivia categories. This feature personalizes the gaming experience and empowers players to tailor the content to their interests, increasing the replayability factor. Players can input any category, and the AI generates relevant questions, making each session distinct and engaging.

Procedural Content Generation:

To address the high costs of content development, TrivAi uses procedural content generation techniques. By algorithmically creating game elements such as questions and scenarios, the game can provide a wide range of content without the need for extended manual creation. This method ensures a rich and varied game experience while keeping development costs under control.

Therefore, the problem of replayability is solved in TrivAI by implementing OpenAi's GPT-4o. This AI serves as the backbone of TrivAi's question generation system, using its vast knowledge base and natural language processing capabilities. The following section will focus on reasons why OpenAi's GPT is most suited for the given requirements.

4.1.1. The Benefits of Using OpenAI's GPT

Contextual Memory:

OpenAI's GPT model excels in its ability to recall and maintain context across conversational interactions. This feature helps in reducing question repetition by maintaining context within a session, enhancing the user experience, and ensuring that trivia questions remain fresh and engaging. By intelligently tracking past questions and player responses, TrivAi can dynamically adjust its question-generation process to avoid repetition. This personalized approach enhances the player experience by presenting a diverse array of trivia topics and eliminating the frustration of encountering the same questions multiple times.

Diverse Knowledge Base:

OpenAI's GPT-4o is trained on a vast and diverse dataset that covers a wide range of topics. This extensive knowledge base allows TrivAi to generate questions on virtually any subject matter, ensuring a rich and varied gaming experience. The ability to tap into such a comprehensive knowledge repository means that the trivia questions can span multiple domains, from history and science to pop culture and sports, catering to the diverse interests of players.

- **Natural Language Processing Capabilities**: GPT-4o's advanced natural language processing capabilities enable it to generate human-like text that is coherent and contextually relevant. This is essential for creating trivia questions that are not only accurate but also engaging and easy to understand. The high quality of text generation ensures that the questions are clear and free of ambiguity, which is crucial for maintaining player interest and satisfaction.
- **Versatility:** GPT-4o's ability to generate content in real time without needing extensive manual updates is a significant advantage for maintaining engagement. This capability allows TrivAi to offer a continuously evolving gameplay experience. The AI's versatility means that developers can focus on other critical aspects of the game while relying on GPT-4o to handle content generation efficiently.

Comparison with Other AI Solutions:

- **Google Cloud Natural Language API**: While excellent for text analysis and sentiment detection, it lacks the advanced text generation capabilities of GPT-3.

- **IBM Watson**: Offers robust NLP services but is more complex to integrate and does not match GPT-4o's versatility in text generation.
- **Microsoft Azure Cognitive Services**: Provides a comprehensive suite of AI tools, but its text generation capabilities are not as advanced as GPT-4o.
- **Bing AI**: Known for search and recommendation capabilities, but does not provide the same level of contextual and diverse text generation as GPT-4o.

By using OpenAI's GPT-4o for dynamic question generation, TrivAi effectively addresses the challenge of replayability. The AI's contextual memory, diverse knowledge base, advanced natural language processing capabilities, and versatility in real-time content generation ensure that each game session delivers new, captivating content tailored to players' interests. This strategic choice not only boosts the player experience but also simplifies maintenance and content development processes, making it a better solution than other available AI technologies.

4.2. Maintenance and Development Cost Solutions

By integrating OpenAI's GPT-API for real-time question generation, TrivAi significantly reduces maintenance and development costs. This approach minimizes the primary maintenance task of ensuring that API calls remain functional and up to date, which requires minimal effort compared to traditional content creation and updates. As a result, the need for ongoing content updates and maintenance is significantly reduced.

One of the key advantages of OpenAI's GPT-API is its ease of implementation. The API is designed to be developer-friendly, with comprehensive documentation and resources available to assist in the integration process. This ease of use allows for a smoother implementation and quicker deployment of AI-driven solutions.

Furthermore, OpenAI is committed to ongoing research and development, continuously improving its API's capabilities and performance. This commitment to innovation ensures that TrivAi can benefit from the latest advancements in artificial intelligence technology, further enhancing the quality and effectiveness of its question-generation system.

4.3. Platform-Specific Development Solutions

Cross-Platform Development Tool: TrivAI was developed using Unity, a leading game development platform, to ensure cross-platform compatibility, including Android, iOS, Windows, and more. By utilizing Unity's strong cross-platform capabilities, TrivAI is minimizing the need for platform-specific adjustments, making the development process simpler, and ensuring a consistent experience for players across devices and operating systems. This approach allows TrivAI to reach a wider audience.

4.3.1. The Advantages of Unity

Cross-Platform Compatibility: Unity stands out for its seamless cross-platform capabilities. Unlike some other engines that may focus primarily on one platform, Unity enables developers to create games that run on a wide variety of platforms, including Android, iOS, Windows, macOS, Linux, and even web browsers. This versatility ensures that TrivAI can reach a broad audience without the need for extensive reworking for different devices.

Ease of Use: Unity is known for its user-friendly interface and comprehensive documentation. This makes it accessible not only to seasoned developers but also to those who are newer to game development. The extensive tutorials, sample projects, and a large community of users make learning and troubleshooting easier. This ease of use helps streamline the development process for TrivAI.

Asset Store: Unity's Asset Store is a significant advantage, offering a vast library of ready-made assets, tools, and plugins. These resources can drastically reduce development time and costs by providing pre-built components that can be integrated into TrivAI. This allows developers to focus more on unique game features and content rather than building everything from scratch.

Active Community and Support: Unity boasts one of the largest and most active developer communities. This means that finding solutions to problems, getting advice, and accessing a wealth of shared knowledge is easier. Additionally, Unity provides comprehensive support through its forums, documentation, and customer service, which is crucial for maintaining and updating TrivAI effectively.

Regular Updates and Improvements: Unity is continuously updated with new features, bug fixes, and improvements. This ongoing development ensures that Unity remains at the cutting edge of technology, providing TrivAI with access to the latest advancements in game development. This future-proofs the game, ensuring it can incorporate new technologies and stay competitive.

Comparison with Other Engines:

- **CryEngine**: Offers advanced rendering capabilities and visual fidelity but lacks extensive documentation and community support.
- **Godot Engine**: Lightweight and open-source, suitable for indie developers, but may lack advanced features found in Unity.
- **Amazon Lumberyard**: Integrates seamlessly with Amazon Web Services, but with limited community and plugin support compared to Unity.
- **GameMaker Studio**: Known for its simplicity and ease of use, making it ideal for beginners and 2D game development, but may lack advanced features and scalability compared to Unity.

By using Unity for cross-platform development, TrivAi effectively addresses the challenge of platform compatibility. Unity's powerful engine, extensive asset store, strong community support, and robust cross-platform capabilities ensure that the game delivers a consistent and engaging experience across various devices. This strategic choice not only enhances the player experience but also simplifies the development process and reduces costs, making it a superior solution compared to other available game engines.

4.4. Cross-Device Progress Retention Solution

To address the issue of cross-device progress retention, TrivAi utilizes PlayFab, a comprehensive backend platform for live games. PlayFab provides robust cloud-based data storage and synchronization, allowing players to save their progress and access it seamlessly across multiple devices. By leveraging PlayFab's user authentication and real-time data handling, TrivAi ensures that player progress is consistently updated and securely stored in the cloud. This solution eliminates data conflicts and loss, providing a smooth and uninterrupted gaming experience for users switching between devices. Additionally, PlayFab's scalable infrastructure efficiently

handles user data, making it easier for developers to manage and maintain cross-device progress retention.

Implementing PlayFab is straightforward and requires minimal effort, allowing developers to focus on enhancing the gaming experience rather than dealing with complex backend integrations.

4.4.1. Advantages of PlayFab Integration

PlayFab was chosen for TrivAi's cross-device progress retention solution due to its robust and developer-friendly features. Here are the key reasons:

- Comprehensive Data Management: PlayFab offers a powerful and flexible cloud-based data storage system that securely stores user progress and game data. This ensures that player progress is saved and synchronized across all devices seamlessly.
- User Authentication: PlayFab provides robust user authentication mechanisms, supporting various login methods such as email, social media accounts, and device-based logins. This ensures that players can easily access their accounts and continue their progress on any device.
- Real-Time Data Synchronization: With PlayFab, data synchronization happens in real time, meaning that any progress or changes made by the player are instantly updated across all devices. This feature prevents data conflicts and ensures a smooth gaming experience.
- Scalability: PlayFab's scalable infrastructure is capable of handling a large number of users and data requests simultaneously. This is crucial for maintaining performance and reliability as the user base grows.
- Ease of Implementation: PlayFab is designed to be developer-friendly, with extensive documentation and support. It integrates easily with popular game development engines like Unity, enabling quick and efficient implementation. This allows developers to focus more on game development rather than backend complexities.

Comparison with Other Solutions

- Firebase: Google's Firebase offers robust real-time database capabilities and easy integration with various Google services, but it can become expensive as your app

scales. It's worth noting that Firebase's SDK compatibility with Unity can be problematic at times, potentially causing challenges in integration and development workflows.

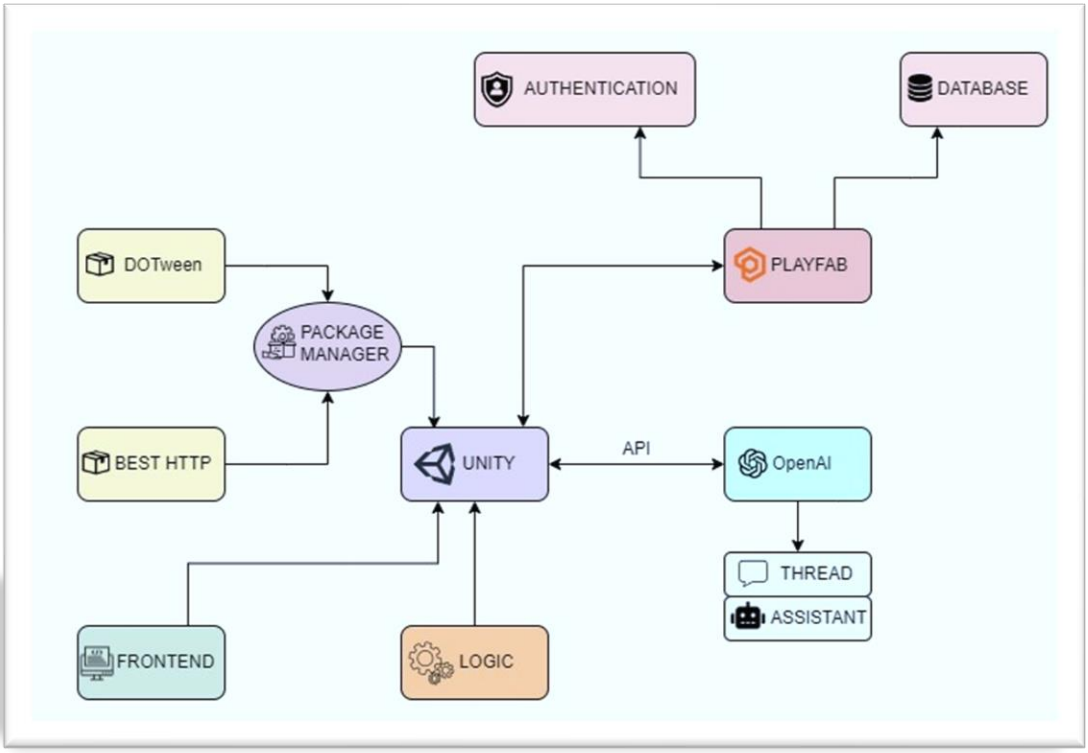
- AWS GameLift: Amazon’s GameLift provides powerful backend support with scalable server infrastructure, excellent for handling multiplayer game sessions and real-time data, but it is more complex to set up and manage.
- GameSparks: GameSparks provides comprehensive backend services, including social and multiplayer features, real-time analytics, and leaderboards, but uncertainties in long-term support and development may arise due to its acquisition by Amazon.

The solutions implemented in TrivAi address significant challenges in game development, such as maintaining player engagement, managing costs, creating content, and ensuring compatibility across various platforms. By using artificial intelligence to generate dynamic content, adopting efficient development practices, and using cross-platform tools, TrivAi delivers a durable and addictive gaming experience. The next chapter will delve into the implementation details and the impact of these solutions on the overall game design and player experience.

5. Approach and Implementation

In this chapter, we delve into the technical implementation of TrivAI, focusing on the specific tools and technologies utilized to bring the project to life.

To provide a clearer understanding, the following diagram illustrates the interconnected components of TrivAi and their roles within the project:



Structure of the project “TrivAI”

Made with: https://app.diagrams.net

As can be seen, the core of TrivAI is built on Unity, a versatile game engine platform. Unity's flexibility and extensive ecosystem made it the ideal choice for developing this Trivia Game.

From the central hub of Unity, we integrated various other technologies to handle different aspects of the game. PlayFab was chosen for its powerful backend services, including database management and user authentication, ensuring that player data is securely stored and easily accessible across devices. For generating dynamic trivia questions and managing conversations,

we integrated OpenAI’s API, leveraging its advanced natural language processing capabilities to provide an engaging and seamless user experience.

Additionally, we utilized Unity's Package Manager to incorporate essential packages that streamlined the development process and enhanced the game's functionality. The front-end development was meticulously crafted to ensure an intuitive and appealing user interface, while the game logic was designed to provide a smooth and enjoyable gameplay experience.

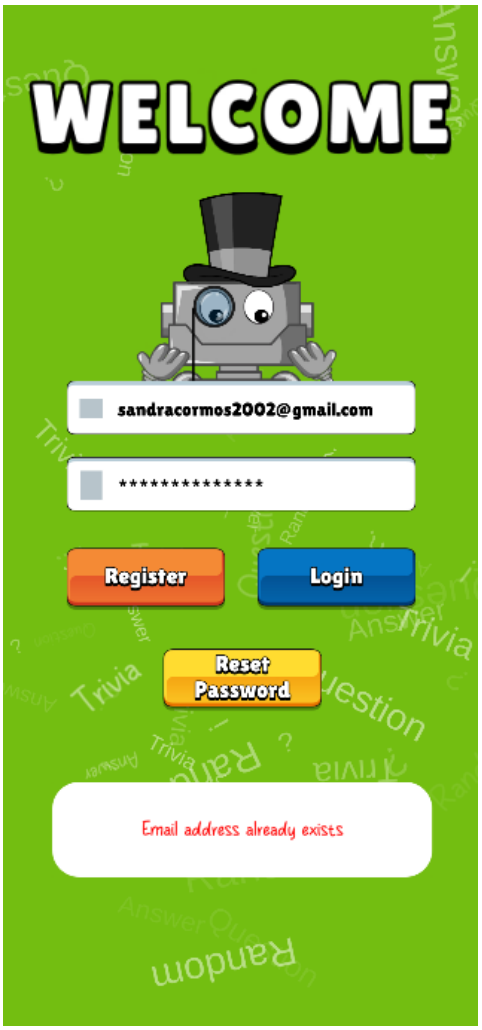
In the sections that follow, we will explore each of these components in detail, discussing the implementation strategies.

5.1 Design of the Application

The design of TrivAI combines vibrant visuals with functional elements to create an engaging and user-friendly experience. This chapter explores the technical aspects of the application's design, including the tools and components used, the adaptability of the interface, and the interactive features. Screenshots are provided to illustrate key components and user interface elements.

5.1.1 Authentication Page

The Authentication Page serves as the entry point for users, designed with engaging visuals and interactive features.



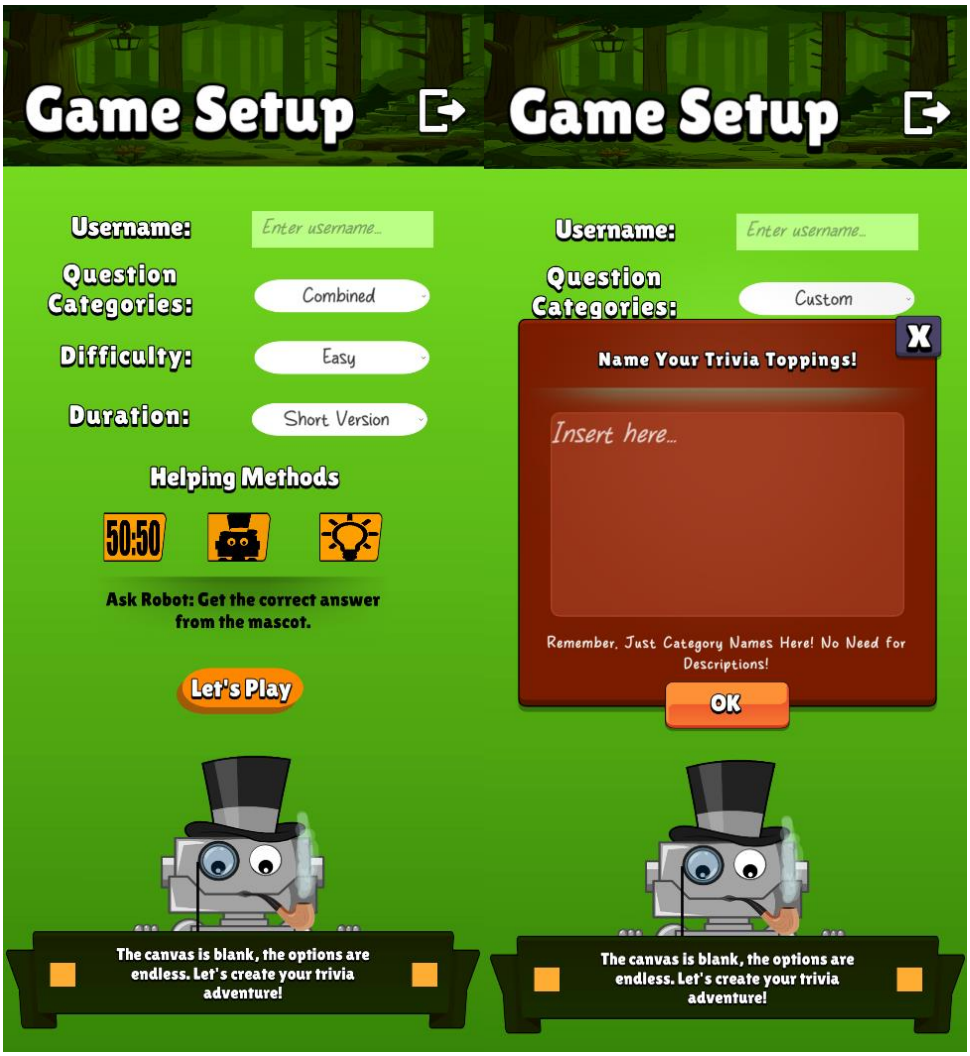
Authentication Page in TrivAI

- **Robot Mascot:** TrivAI features a custom-drawn robot mascot that serves as the game's visual centerpiece. This character enhances user engagement by interacting with users, such as covering its eyes when passwords are entered or following the user's movements on the screen with its eyes, adding a whimsical touch to privacy features. The robot is integrated into various screens, including the login and game setup screens.
- **Real-Time Validation:** The email input field features real-time validation, implemented through Unity's UI InputField component and C# scripting. This feature checks the email format as the user types, providing immediate feedback if the input is invalid, thereby reducing user errors and improving the overall user experience.

- **Animations:** The page includes animations created with DoTween, such as the robot following the cursor or touch movements and placing its hands over its eyes when the password field is selected. Additionally, words float randomly in the background, creating an engaging and dynamic environment.

5.1.2 Game Setup Screen

The Game Setup Screen allows users to tailor their gameplay experience. It features several interactive elements that enhance customization and user engagement.



Question Categories

One notable feature is the ability to customize question categories. Users are presented with two options: "Custom Categories" and "Combined Categories". With "Custom Categories", users can input their preferred category names, allowing for a personalized trivia experience. Alternatively,

"Combined Categories" randomly selects three categories from a diverse pool of 100 options, ensuring variety and excitement with each game session.

Difficulty Levels

Difficulty levels further enhance gameplay dynamics, offering three distinct levels: easy, medium, and hard. Each level is meticulously integrated into the question requests, adjusting the challenge level accordingly.

Game Duration

The duration of the game is another customizable element, offering three choices: short, medium, and long. These durations dictate the number of questions per game session, catering to users' time preferences and attention spans.

Customizable Username

Furthermore, users have the option to input their desired username, adding a personal touch to their gaming experience. In the absence of user input, the system assigns a random username for the current game session.

Help Methods

The inclusion of toggle switches for help methods provides users with additional assistance during gameplay. Each help method, such as Fifty-Fifty, Ask Robot, and Ask for Tip, is accompanied by clear instructions. This empowers users to make informed decisions and maximize their chances of success.

- **Fifty-Fifty:** Eliminates two incorrect answers, leaving one correct and one incorrect option.
- **Ask Robot:** Provides the correct answer.
- **Ask for Tip:** This feature offers a hint for the current question, guiding users towards the correct answer without giving it away outright, generated from the same request as the question and answers.

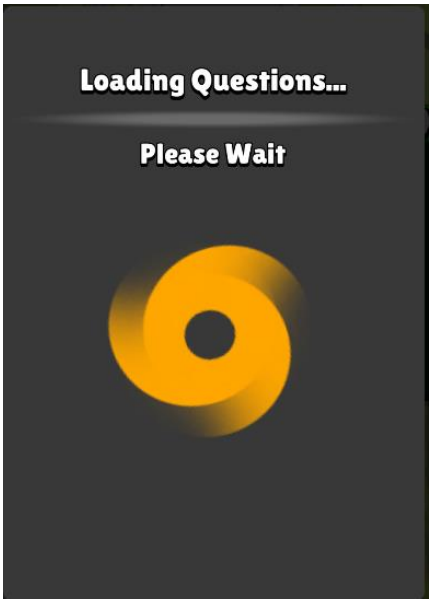
Exit Game

Lastly, a prominent logout button offers users the flexibility to exit the game at any point. With a single click, users can seamlessly transition back to the authentication page

In summary, the Game Setup Screen epitomizes TrivAI's commitment to user-centric design, offering a comprehensive suite of customization options tailored to individual preferences. From

personalized question categories to adjustable difficulty levels, every aspect of the game setup is meticulously crafted to ensure a captivating gaming experience.

5.1.3 Loading Screen



The Loading Screen appears while the game prepares questions and other data.

- **Animations:** During this time, animations created with DoTween keep users engaged. In the event of slow API responses or connection issues with OpenAI, an error screen will appear, redirecting users back to the game setup menu for another attempt.

5.1.3 Gameplay Screen

The Gameplay Screen is the heart of the TrivAI experience. It displays trivia questions and multiple-choice answers, designed to keep users engaged and challenged.

Question Display

Each question appears alongside four multiple-choice answers, inviting users to test their knowledge and wit. The user selects an answer, and feedback is provided immediately.

Relaxed Gameplay Experience: A notable feature of the Gameplay Screen is the absence of a timer, allowing users to engage with questions at their own pace without the pressure of time constraints. This design choice promotes a relaxed and enjoyable gaming experience, fostering deeper engagement with the content.

Help Methods

The user can use the help variants at any time during the game but only once per match. When starting a new match, they will be reset. These help methods are represented by buttons with an icon representing each type.

Scoring and Leveling

Users earn experience points (XP) for correct answers, contributing to their overall score and level progression. The leveling system is designed to keep players motivated, with XP thresholds increasing as they advance in levels. For each correct answer, users earn 100 XP. The XP required for leveling up is displayed in the corner of the screen, along with the level and current XP. After each level up, the number of XP required for the next level increases by 150. The leveling system is designed to motivate users to progress in the game.

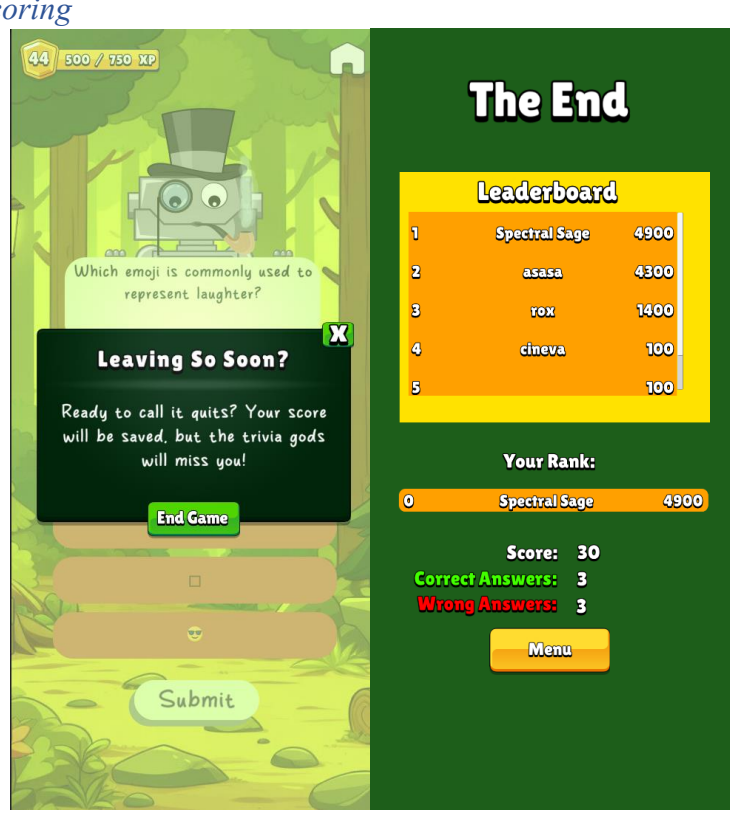
Leave Game Functionality

A button allows users to end the game prematurely. Their score is saved, and they are redirected to the end-game statistics menu.

5.1.4. End Game Statistics Menu

The End Game Statistics Menu provides users with an overview of their performance and standings.

Leaderboard and Scoring



At the end of each match, the leaderboard is prominently displayed, showing the user's ranking among all players. This feature fosters a competitive environment, encouraging users to improve their scores and climb the ranks. The leaderboard logic includes:

- **Real-Time Updates**: The leaderboard is updated in real time, reflecting the latest scores and rankings.
- **Scrolling View**: The leaderboard is presented in a scroll view, allowing users to easily navigate through the rankings and see how they compare with others.

Detailed Statistics

Users can see their total score, correct answers, and incorrect answers, providing a comprehensive overview of their performance.

The formula for calculating the score in TrivAI is designed to ensure that both the current level and the accumulated XP are considered, giving more weight to the level.

Back To Menu

From the end game statistics menu, users can return to the game setup menu to start a new session or adjust their preferences by just clicking a button.

5.2. Technical Implementation

Unity Canvas:

The core UI of TrivAI is built on Unity's canvas system, providing a scalable and adaptable interface. Each UI element is anchored and positioned relative to the canvas, ensuring it adjusts correctly to various screen sizes and orientations.

TextMeshPro:

TextMeshPro is utilized for all textual elements, offering high-quality text rendering and extensive customization capabilities. This tool ensures that all text within TrivAI is clear and legible, enhancing the user experience across different devices.

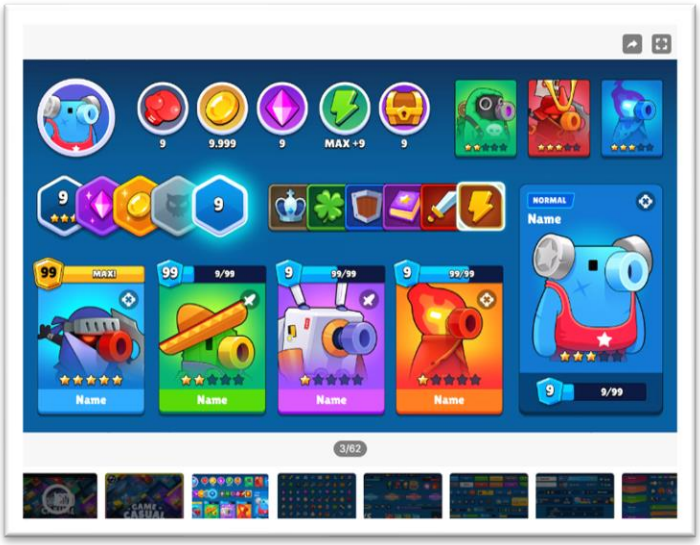
C# Scripting:

The functionality of interactive elements, real-time validation, game mechanics, and user preferences is managed through C# scripts. These scripts handle user inputs, validate data, manage game state, and update the UI dynamically, ensuring a responsive and interactive user experience.

The design of TrivAI integrates custom artwork, advanced Unity components, and responsive layout techniques to deliver a high-quality user experience.

Imported Assets:

To enhance the visual appeal and usability of interactive elements, "TrivAI" incorporates button assets imported from the Unity Asset Store. These pre-designed buttons offer a polished look and feel, contributing to the overall aesthetic of the app. The buttons are integrated into the UI, ensuring a consistent user experience across different screens.



Asset name: GUI Pro - Casual Game

By leveraging Unity's powerful UI tools and ensuring responsive design, TrivAI achieves a professional and well-rounded with attention to details interface that is functional and visually appealing. The appeal of casual games lies in their broad accessibility, and TrivAI targets a diverse audience, including those who may not consider themselves traditional gamers. Its simplicity and universal themes make it attractive to users of all ages and gaming experience levels, ensuring it remains engaging and enjoyable for a wide range of players.

5.3. PlayFab

Playfab is a complete backend platform for games, that features a wide range of services, from real-time analytics to player data management, in-game economy systems, and multiplayer services.

Acquired by Microsoft in 2018, PlayFab provides game developers with a robust and scalable infrastructure, allowing them to manage their games more efficiently and effectively.
PlayFab's ease of implementation and comprehensive documentation made it a natural choice for handling user authentication and data management in TrivAI. The platform's robust security features ensure that player data is both secure and accessible across different devices, facilitating a seamless gaming experience.

PlayFab Setup is straightforward, offering seamless integration into any project

- Account Creation
- Title Creation: Once logged in it is necessary to create a new title within the PlayFab dashboard. This title serves as the central hub for managing the game's backend services, including authentication.
- Unity Integration: Importing the PlayFab SDK into your Unity project involves downloading the SDK package from the PlayFab website or the Unity Asset Store and importing it into the project.
- Initialization: The last step in initializing the PlayFab SDK within the Unity project is performed by making the necessary configurations

5.3.1. Authentication

Login

For Authentication TrivAI uses a recoverable login mechanism. A recoverable login mechanism requires some identity information from the player. In this case, the authentication is made directly with PlayFab through an API call, by using an email address and a password.

After the user submits their login credentials in the specific input fields through the UI, the script linked to it will then make an API call to PlayFab's authentication service to verify the credentials and log the player in.

This API call is constructed using the HTTP POST method, and is directed towards PlayFab's authentication endpoint, typically represented by '/Client/LoginWithEmailAddress'.

The payload of this HTTP request is formatted as a JSON object, containing the user's email address and password in a structured manner. Here's an example of the payload for the login method:

```
0 references
public void LoginButton()
{
    var request = new LoginWithEmailAddressRequest
    {
        Email = emailInput.text,
        Password = passwordInput.text,
    };
    PlayFabClientAPI.LoginWithEmailAddress(request, OnLoginSuccess, OnError);
}
```

Within the payload:

- "Email" corresponds to the user's provided email address.
- "Password" contains the user's password.

This JSON payload is securely transmitted over HTTPS, ensuring encryption of sensitive data during transit. Through this streamlined process, TrivAI ensures a seamless and secure login experience for its users, leveraging the robust infrastructure provided by PlayFab's authentication service.

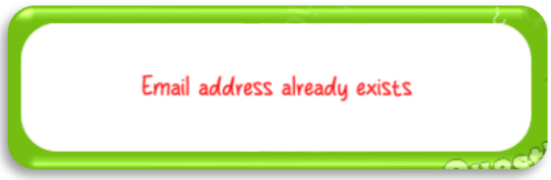
If the provided credentials align with an existing player account within the PlayFab ecosystem, the authentication service facilitates the successful login of the player into the TrivAI platform. Conversely, in scenarios where the provided credentials fail to match any existing account or encounter other authentication-related issues, PlayFab's authentication service communicates relevant error messages back to the invoking script. The script, equipped with error-handling mechanisms, interprets the response from PlayFab's authentication service. In the event of authentication failure, the user will be notified of the encountered issue via the UI, where an error will appear, specifying the nature of the problem.

Registration

Similarly, for user registration, the implementation method follows a similar approach. When a user submits their registration details, an associated script triggers an API call to PlayFab's

registration service. This API call is constructed using the HTTP POST method. It is directed towards PlayFab's registration endpoint, represented by ‘Client/RegisterPlayFabUser’.

However, if the provided email address is already registered or other registration-related issues occur, PlayFab's registration service communicates relevant error messages back to the invoking script. Similar to the authentication process, the script is equipped with error-handling mechanisms to interpret the response from PlayFab's registration service.



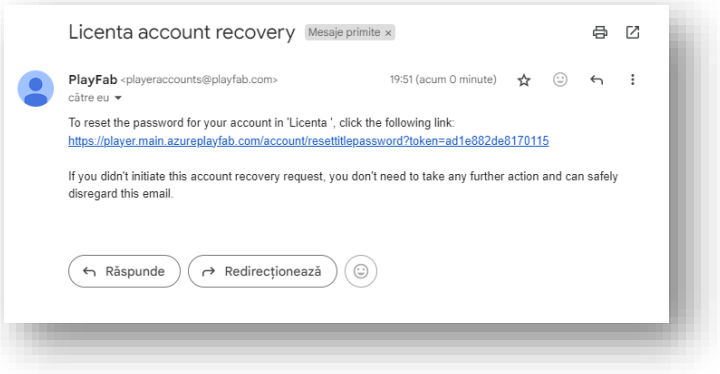
Example of Registration Error

TrivAI

Password Recovery

TrivAI also provides a password recovery feature for users who have forgotten their passwords. A button in the UI triggers a script that calls an API from PlayFab specifically designed for password recovery. This API call is directed toward PlayFab's password recovery endpoint and follows a similar structure to other API calls in the system.

This service sends a recovery email to the user's registered email address, containing a link with a token.



Example of Recovery Email

TrivAI

Upon clicking the link, the user is redirected to a page within TrivAI's interface where they can input a new password. Once the new password is submitted, it is securely saved within the system.

5.3.2. Database

TrivAI uses PlayFab's powerful database infrastructure to store a variety of user data essential to delivering a personalized gaming experience. Key information stored includes user credentials, player profiles, game progress, achievements preferences, settings, player levels, scores, and leaderboard rankings.

When a new user registers on TrivAI, their credentials, such as email addresses and passwords, along with initial profile information like usernames and other IDs that are created for OpenAI purposes, are saved in the PlayFab database.

Saving Data

Registration and Initialization:

After the successful registration of a player, some relevant identifiers will be generated for further use. This data is sent via a POST request to PlayFab's /UpdateUserData API endpoint.

```
public void SaveUsersOpenAiIdData()
{
    var request = new UpdateUserDataRequest
    {
        Data = new Dictionary<string, string>
        {
            {"ThreadId", References.Instance.OpenAi.ThreadId },
            {"AssistantId", References.Instance.OpenAi.AssistantId },
            {"Username", References.Instance.Username }
        }
    };
    PlayFabClientAPI.UpdateUserData(request, OnDataSend, OnError);
}
```

This method, SaveUsersOpenAiIdData, is used to store user-specific data within PlayFab's database. The method constructs an UpdateUserDataRequest with key-value pairs representing the data to be stored. The PlayFabClientAPI. The UpdateUserData function then sends this data to PlayFab, handling both successful and error responses with designated callback methods (OnDataSend and OnError).

Game Progress:

As users play TrivAI, their game progress, scores, and player levels are periodically saved. This typically happens at significant points such as the end of a level or the end of a game. By saving this data, TrivAI ensures that users can resume their game from where they left off, providing a consistent and engaging experience.

```
1 reference
public void SaveUsersGameDataOnEndGame(int level, int score)
{
    Debug.Log("data sent to playfab");
    var request = new UpdateUserDataRequest
    {
        Data = new Dictionary<string, string>
        {
            {"Level", level.ToString() },
            {"Score", score.ToString() }
        }
    };
    PlayFabClientAPI.UpdateUserData(request, OnUserDataSuccessfullySendOnEndGame, OnUserDataSendOnEndGameFailure);
}
```

Player Statistics:

To update player statistics on the leaderboard, TrivAI employs a method called SendLeaderboard. This method is responsible for updating the player's score on the leaderboard. It constructs an UpdatePlayerStatisticsRequest object containing the player's score, which is then sent to PlayFab using the PlayFabClientAPI.UpdatePlayerStatistics method. This method sends the request to PlayFab, where it is processed, and the leaderboard is updated accordingly.

```
1 reference
public void SendLeaderboard(int score)
{
    var request = new UpdatePlayerStatisticsRequest
    {
        Statistics = new List<StatisticUpdate>
        {
            new StatisticUpdate
            {
                StatisticName = "PlatformScore",
                Value = score
            }
        }
    };
    PlayFabClientAPI.UpdatePlayerStatistics(request, OnLeaderboardUpdate, OnError);
}
```

Data Retrieval

Retrieving User Data:

To retrieve user-specific data, such as OpenAI thread IDs and assistant IDs, a GET request is made to PlayFab's /GetUserData API endpoint. This is typically done during user login to initialize their session with relevant data.

```
2 references
public void GetUsersOpenAiId()
{
    PlayFabClientAPI.GetUserData(new GetUserDataRequest(), OnDataReceived, OnError);
}

1 reference
void OnDataReceived(GetUserDataResult result)
{
    if(result.Data is not null && result.Data.ContainsKey("ThreadId") && result.Data.ContainsKey("AssistantId") &&
       !string.IsNullOrEmpty(result.Data["ThreadId"].Value) && !string.IsNullOrEmpty(result.Data["AssistantId"].Value) )
    {
        References.Instance.OpenAi.ThreadId = result.Data["ThreadId"].Value;
        References.Instance.OpenAi.AssistantId = result.Data["AssistantId"].Value;
    }
    else
    {
        Debug.Log("[PlayFabManager] data not found");
    }
}
```

Retrieving Users Statistics:

Similar to the step of user data retrieval, is approached users statistics retrieval only that a GET request is made to PlayFab's /GetPlayerStatistics API endpoint.

```
1 reference
public void GetStatistics()
{
    PlayFabClientAPI.GetPlayerStatistics(
        new GetPlayerStatisticsRequest(),
        OnGetStatistics,
        error => Debug.LogError(error.GenerateErrorReport())
    );
}

1 reference
void OnGetStatistics(GetPlayerStatisticsResult result)
{
    Debug.Log("Received the following Statistics:");
    foreach (var eachStat in result.Statistics)
    {
        initialScore.SetText(eachStat.Value.ToString());
    }
}
```

These scores foster a competitive environment and encourage the community to engage by allowing users to compare their achievements with others, making the game more dynamic and interactive

By leveraging PlayFab's comprehensive API for data storage and retrieval, TrivAI ensures that all critical user data is securely managed.

5.4. OpenAI

In this chapter, we explore the integration of OpenAI into TrivAI to handle dynamic content generation, particularly focusing on the creation of trivia questions. Leveraging OpenAI's GPT-4o API, we have implemented a system that not only generates unique and contextually

appropriate questions but also enhances the overall user experience by minimizing repetition and maintaining engagement.

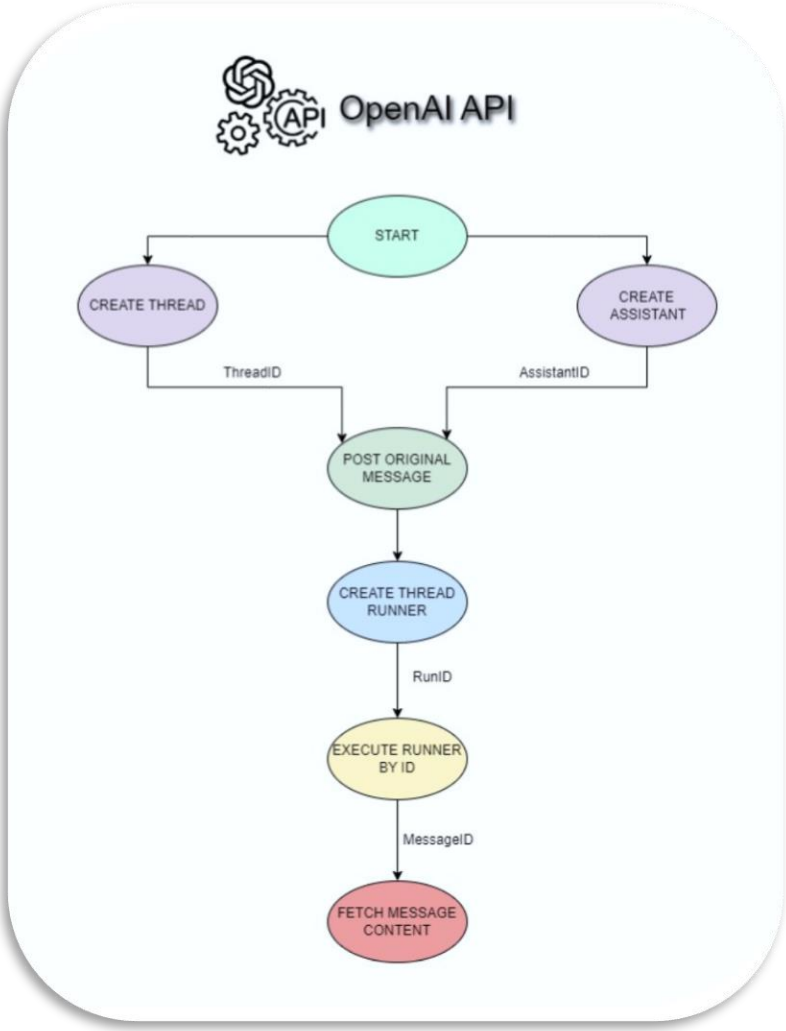
The following sections will provide an in-depth look into why OpenAI was chosen, how it was integrated, and the specific benefits it brings to the project.

5.4.1. Integration Process

To effectively integrate OpenAI into TrivAI, the following steps were undertaken:

- **API Setup:** Access to the OpenAI GPT-4o API was set up by creating an account, obtaining API keys, and configuring the development environment.
- **Dynamic Question Generation**: A system was developed to request and receive trivia questions in real time using the API, ensuring each game session presents a unique set of questions.

For a better understanding of how the dynamic question generation was implemented, the following diagram was created:



OpenAI API configuration used in TrivAI

Made with: https://app.diagrams.net

For starters, every user needs to have a unique ThreadID and a unique AssistantID. This step is very important because thanks to it we do not have repeatability in questions.

Creating a Thread with OpenAI's API

To effectively manage conversations within TrivAI, it is essential to create and maintain threads using OpenAI's API. Therefore the BestHTTP library is used to handle HTTP requests due to its comprehensive features and seamless integration with Unity. This section shows the implementation of the “CreateThread” class, which facilitates conversations by interacting with the OpenAI API.

Requesting the Thread ID

The “PostThread” method is responsible for sending a POST request to the OpenAI API to create a new conversation thread. This method encapsulates the request construction, header configuration, and the actual transmission of the request using the BestHTTP library. Below is the method implementation followed by an explanation of its components:

```
2 references
public void PostThread()
{
    if (string.IsNullOrEmpty(PostUriString))
        return;

    HTTPRequest request = new HTTPRequest(new Uri(PostUriString), HTTPMethods.Post, OnPostRequestFinished);
    request.SetHeader("Content-Type", "application/json");
    request.SetHeader("Authorization", References.Instance.OpenAi.ApiKey);
    request.SetHeader("OpenAI-Beta", "assistants=v1");
    request.Send();
    Debug.Log($"[CreateThread] Request sent!");
}
```

The “PostThread” method in the “OpenAiCreateThread” class initiates a new thread by interacting with the OpenAI API. Essential headers like Content-Type (application/json), Authorization (API key), and OpenAI-Beta (custom functionality) ensure proper formatting, secure access, and activation of required features. By incorporating these headers into the HTTP request, the “PostThread” method ensures that communication with the OpenAI API is properly structured and authenticated. This careful configuration facilitates the smooth transmission of data and commands, eventually contributing to the successfully creation of conversation threads within the application.

Receiving the Thread ID

The “OnPostRequestFinished” method is a callback function in the “OpenAiCreateThread” class. It handles the response from the HTTP POST request sent to the OpenAI API. This method uses a switch statement to manage different possible states of the HTTP request and perform appropriate actions based on the outcome.

Success Handling:

```
1 reference
private void OnPostRequestFinished(HTTPRequest req, HTTPResponse resp)
{
    Debug.Log($"[CreateThread] Response received !");
    postResponseData = resp.DataAsText;
    switch (req.State)
    {
        // The request finished without any problem.
        case HTTPRequestStates.Finished:
            if (resp.IsSuccess)
            {
                Debug.Log($"[CreateThread] Response received with data: {resp.DataAsText}");
                openAiCreateThreadResponse = JsonConvert.DeserializeObject<OpenAiCreateThreadResponse>(resp.DataAsText);
                References.Instance.OpenAi.ThreadId = openAiCreateThreadResponse.id;
                if (!string.IsNullOrEmpty(References.Instance.OpenAi.ThreadId) && !string.IsNullOrEmpty(References.Instance.OpenAi.AssistantId)) {
                    PlayFabManager.Instance.SaveUsersOpenAiId();
                }


                //Post initial message to thread
                References.Instance.OpenAi_OriginalMessagePoster.PostMessage();

                //Attach assistant on thread and create a runner
                References.Instance.OpenAi_CreateAssistantThreadRunner.TryToRunAssistantOnThread();

                //Execute Runner
                References.Instance.OpenAi_ExecuteThreadRunner.TryExecuteRunner();
                References.Instance.OpenAi_ResultMessageFetcher.TryFetchingMessageContent();
            }
```

- If the response indicates success (resp.IsSuccess), the response data is deserialized into an OpenAiCreateThreadResponse object.
- The ThreadId is extracted from the response and stored in a singleton reference.
- The ThreadId is a crucial piece of information as it uniquely identifies the newly created thread. This ID will be used for further interactions with the thread, such as posting messages or fetching results.
- The method proceeds to post an initial message to the thread, attach an assistant to the thread, execute a runner, and fetch message content.

Error State:

```
// The request finished with an unexpected error. The request's Exception property may contain more info about the error.
case HTTPRequestStates.Error:
    Debug.LogError($"[OpenAiCommunicator] Request Finished with Error! " +
                   (req.Exception != null ? (req.Exception.Message +
                   "\n" + req.Exception.StackTrace) : "No Exception"));
    break;
```

- Purpose: This block handles the case where the request finished with an error.
- Action: Logs an error message, including exception details if available.

Aborted State:

```
// The request aborted, initiated by the user.
case HTTPRequestStates.Aborted:
    Debug.LogWarning($"[OpenAiCommunicator] Request Aborted!");
    break;
```

- Purpose: This block handles the case where the request was aborted by the user.
- Action: Logs a warning that the request was aborted.

Connection Timed Out State:

```
// Connecting to the server is timed out.
case HTTPRequestStates.ConnectionTimedOut:
    Debug.LogError($"[OpenAiCommunicator] Connection Timed Out!");
    break;
```

- Purpose: This block handles the case where the connection to the server timed out.
- Action: Logs an error message indicating the connection timed out.

Request Timed Out State

```
// The request didn't finished in the given time.
case HTTPRequestStates.TimedOut:
    Debug.LogError($"[OpenAiCommunicator] Processing the request Timed Out!");
    break;
```

- Purpose: This block handles the case where the request did not finish in the allotted time.
- Action: Logs an error message indicating the request timed out.

This careful orchestration enables the creation of conversation threads, forming the backbone of dynamic and interactive user experiences within TrivAI.

Creating an Assistant with OpenAI's API

Creating an assistant thread is a critical aspect of managing conversations within TrivAI. Similar to creating a general thread, this process involves sending a POST request to the OpenAI API, but with specific configurations tailored for assistant integration. This section outlines the

implementation of the “CreateAssistantThread” function, which ensures the setup of an assistant within a conversation thread.

Requesting the Assistant ID

The “PostAssistant” method sends a POST request to the OpenAI API to create a new assistant thread. It includes headers such as Content-Type, Authorization, and OpenAI-Beta. The “postData” is included in the request body as JSON.

```
2 references
public void PostAssistant()
{
    if (string.IsNullOrEmpty(PostUriString))
        return;

    HTTPRequest request = new HTTPRequest(new Uri(PostUriString), HTTPMethods.Post, OnPostRequestFinished);
    request.SetHeader("Content-Type", "application/json");
    request.SetHeader("Authorization", References.Instance.OpenAi.ApiKey);
    request.SetHeader("OpenAI-Beta", "assistants=v1");
    request.UploadSettings.UploadStream = new JSonDataStream<OpenAiPostAssistant>(postData);
    request.Send();
    Debug.Log($"[CreateAssistant] Request sent!");
}
```

Receiving the Assistant ID

The “OnPostRequestFinished” method processes responses from the OpenAI API. On success, it deserializes the response to obtain the AssistantId and stores it. If both ThreadId and AssistantId are available, it saves the user's OpenAI ID and runs the assistant within the thread. It logs messages for errors, aborts, connection timeouts, and request timeouts

By implementing the “PostAssistant” and “OnPostRequestFinished methods”, TrivAI ensures a reliable and efficient process for creating and managing assistant threads with OpenAI's API.

The process of creating a ThreadID and AssistantID is a crucial step that occurs only when a new player registers for the game. Both these identifiers are essential for managing the player's interactions and sessions with the OpenAI API. Once generated, both identifiers are stored in PlayFab, ensuring that they can be reused every time the player signs in again.

Key Feature: Memory and Reusing of Identifiers

One of the key features of TrivAI is the reuse of the same ThreadID and AssistantID across multiple sessions. This capability leverages OpenAI's memory feature, which allows the system to remember previous interactions within a thread. By maintaining the same ThreadID for a player, TrivAI ensures that:

- Context Preservation: The AI retains the context of past conversations, allowing for more coherent and relevant responses in ongoing interactions.
- Avoiding Repetition: Previous questions and answers are remembered, preventing the AI from repeating the same questions and enhancing the user experience.

Posting Message on Thread

Now that a ThreadId and an AssistantID have been created and stored, TrivAI can facilitate conversations between players and the AI by posting messages on the established thread.

Sending Messages

Posting messages on the thread involves sending HTTP POST requests to the OpenAI API, containing the message content and necessary identifiers. The “PostMessage” method handles this task, encapsulating the request construction and transmission.

```
2 references
public void PostMessage()
{
    formattedUriString = string.Format(unformattedUriString, References.Instance.OpenAi.ThreadId);

    HTTPRequest request = new HTTPRequest(new Uri(formattedUriString), HTTPMethods.Post, OnPostRequestFinished);
    request.SetHeader("Content-Type", "application/json");
    request.SetHeader("Authorization", References.Instance.OpenAi.ApiKey);
    request.SetHeader("OpenAI-Beta", "assistants=v1");
    request.UploadSettings.UploadStream = new JSonDataStream<OpenAiPostMessageThread>(postData);
    request.Send();
    Debug.Log($"[OpenAiCommunicator] Request sent!");
}
```

Similar to previous HTTP POST requests, “PostMessage” configures the UploadStream to include the postData in the request body, formatted as JSON. The postData comprises a question generation request based on specified categories. Example of postData:

“Generate a unique multiple-choice question with 4 options. Ensure that the questions don't repeat too often. Provide diverse questions for each request. Please indicate which one is correct and send it back in the following format :

```
10 references
public class Question
{
    public int Id;
    public string QuestionName;
    public List<Answer> Answers;
    public string TipForAnsweringQuestion;
}

[System.Serializable]
3 references
public class Answer
{
    public string text;
    public bool isCorrect;
}
```

Give the response in JSON format. Ensure you provide ONLY a JSON string, no other formatting!"

Also, a very important part is when posting a message on a thread in TrivAI, the URL is instrumental in directing the HTTP POST request to the OpenAI API endpoint responsible for handling message creation within the designated thread. The URL serves as the entry point for communication between TrivAI and the OpenAI infrastructure, facilitating the exchange of data and commands. The URL for posting a message on a thread is constructed dynamically to include the ThreadID. This dynamic construction ensures that the message is directed to the correct thread.

Receiving the ResponseID

Upon making the request, the "OnPostRequestFinished" method handles the response. If the request is successful, indicated by resp.IsSuccess, the response JSON is deserialized into an OpenAiMessageToThreadResponse object using Json.NET.

The OpenAiMessageToThreadResponse class is structured as follows:

```
2 references
public class OpenAiMessageToThreadResponse
{
    public string Id;
    public string Object;
    public string Thread_Id;

}
```

After successfully handling the response, the unique identifier (Id) extracted from the response is stored for further use within TrivAI. However, it's important to note that this identifier is not stored in PlayFab. Instead, it is utilized within the application's logic to manage and track conversations on the established thread.

Creating the Thread Runner

Creating the thread runner in TrivAI involves initiating a new run within a specified thread through a POST request directed at the OpenAI API.

```
2 references
public void PostMessage()
{
    //Format UriString
    formattedPostUriString = string.Format(unformattedPostUriString, References.Instance.OpenAi.ThreadId);
    postData.assistant_id = References.Instance.OpenAi.AssistantId;

    HTTPRequest request = new HTTPRequest(new Uri(formattedPostUriString), HTTPMethods.Post, OnPostRequestFinished);
    request.SetHeader("Content-Type", "application/json");
    request.SetHeader("Authorization", References.Instance.OpenAi.ApiKey);
    request.SetHeader("OpenAI-Beta", "assistants=v1");
    request.UploadSettings.UploadStream = new JSonDataStream<OpenAiPostRun>(postData);
    request.Send();
    Debug.Log($"[OpenAiRunThread] Request sent!");
}
```

The URL utilized for this purpose is dynamically constructed to incorporate the ThreadID. This dynamic construction guarantees that the new run is launched within the confines of the designated thread, thus maintaining context and coherence throughout the conversation that the new run is initiated within the context of the designated thread.

Moreover, the body of the POST request plays a crucial role in specifying additional parameters, such as the AssistantID. This parameter identifies the AI assistant responsible for managing the conversation within the thread. By including the AssistantID in the request body, TrivAI ensures that the appropriate assistant is engaged in the dialogue.

Retrieving the ThreadRunnerId

The “OnPostRequestFinished” method handles the response from the HTTP POST request. It updates the postResponseData variable with the response data retrieved from the server.

Using a switch statement, it evaluates the state of the request and executes different actions based on the outcome:

- If the request finishes without any issues, it checks if the response indicates success. If successful, it logs a message indicating that the request-response was received. It then deserializes the response JSON into an OpenAiRunThreadResponse object, extracts the ThreadRunnerId from the response, and updates the corresponding reference. Finally, it tries to execute the thread runner.
- Unsuccessful requests are being treated with the same logic as the previous one

This method effectively manages the response from the HTTP POST request, providing detailed logging for various scenarios such as success, errors, user-initiated aborts, and timeouts.

Executing a Thread Runner by ID

In TrivAI, executing a thread runner by ID involves initiating a GET request to the OpenAI API, targeting the specified runner within a designated thread. Here's how TrivAI handles the execution of a thread runner by ID:

ExecuteRunnerByID Method:

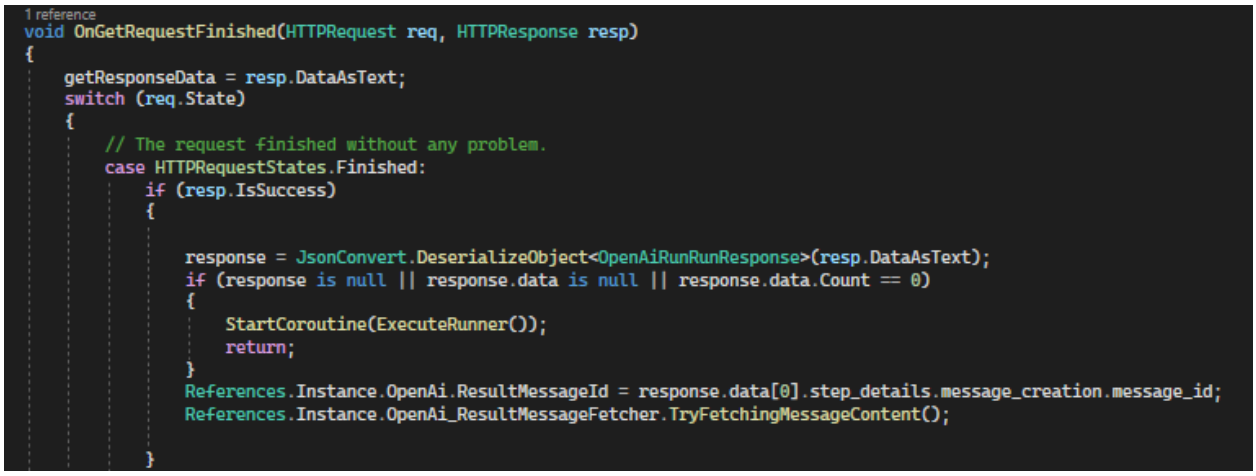
```
2 references
IEnumerator ExecuteRunner()
{
    if (string.IsNullOrEmpty(formattedPostUriString))
        yield break;

    yield return new WaitForSeconds(1);

    HTTPRequest request = new HTTPRequest(new Uri(formattedPostUriString), HTTPMethods.Get, OnGetRequestFinished);
    request.SetHeader("Content-Type", "application/json");
    request.SetHeader("Authorization", References.Instance.OpenAi.ApiKey);
    request.SetHeader("OpenAI-Beta", "assistants=v1");
    request.Send();
    Debug.Log($"[OpenAiCommunicator] Request sent!");
}
```

The ExecuteRunner coroutine method orchestrates the initiation of a GET request to the OpenAI API, specifically targeting the execution of a thread runner within TrivAI. The URL for executing a thread runner by ID is dynamically constructed to incorporate both the ThreadID and the RunnerID.

URL example: “https://api.openai.com/v1/threads/thread_ID/runs/run_ID/steps”

OnGetRequestFinished Method

In the event of a successful response, the method processes the response data by deserializing the JSON using Json.NET for further analysis.



```
1 reference
void OnGetRequestFinished(HTTPRequest req, HTTPResponse resp)
{
    getResponseData = resp.DataAsText;
    switch (req.State)
    {
        // The request finished without any problem.
        case HTTPRequestStates.Finished:
            if (resp.IsSuccess)
            {

                response = JsonConvert.DeserializeObject<OpenAiRunRunResponse>(resp.DataAsText);
                if (response is null || response.data is null || response.data.Count == 0)
                {
                    StartCoroutine(ExecuteRunner());
                    return;
                }
                References.Instance.OpenAi.ResultMessageId = response.data[0].step_details.message_creation.message_id;
                References.Instance.OpenAi_ResultMessageFetcher.TryFetchingMessageContent();

            }
```

Upon successfully parsing the response data, the method checks if the response contains any meaningful data. If the response is empty or lacks essential information, it triggers the execution of the runner again by invoking the “ExecuteRunner” coroutine.

If the response contains relevant data, the method extracts the Message ID from the response. This ID is crucial for tracking and managing messages within the conversation thread.

Once the Message ID is obtained, the method triggers the “TryFetchingMessageContent” function to retrieve the content of the message associated with the extracted ID. This step ensures that the conversation flow continues smoothly, with timely retrieval and processing of message content.

Fetching The Message Content

Lastly, within the “FetchMessageContent” coroutine, a POST request is sent to the OpenAI API with the constructed URI. After a delay of 1 second to ensure smooth processing, the request is sent.

The URL used in this case needs to be properly formatted to include the necessary dynamic values, such as the thread ID and result message ID. In the event of a successful response, the method processes the response data by deserializing the JSON using Json.NET for further analysis.

```
1 reference
void OnPostRequestFinished(HTTPRequest req, HTTPResponse resp)
{
    postResponseData = resp.DataAsText;
    switch (req.State)
    {
        // The request finished without any problem.
        case HTTPRequestStates.Finished:
            if (resp.IsSuccess)
            {
                openAiResponse = JsonConvert.DeserializeObject<OpenAiResponse>(resp.DataAsText);
                if (openAiResponse is null || openAiResponse.content is null || openAiResponse.content.Count == 0 ||
                    openAiResponse.content[0].text is null || string.IsNullOrEmpty(openAiResponse.content[0].text.value))
                {
                    StartCoroutine(FetchMessageContent());
                    return;
                }
                deserializedQuestion = JsonConvert.DeserializeObject<Question>(openAiResponse.content[0].text.value);
                manager.LoadQuestion(deserializedQuestion);
            }
```

If the HTTP request is successful, the method proceeds to handle the response data appropriately. The response JSON data, stored in resp. DataAsText is deserialized into an OpenAiResponse object using Json.NET. This object contains the structured data received from the OpenAI API response.

The method then checks if the openAiResponse object is not null and if it contains valid content. Specifically, it ensures that the content property of the openAiResponse object is not null and contains at least one item and that the text value of the first content item is not null or empty. This validation helps ensure that the response contains the expected data structure and prevents potential errors during further processing.

If the content is empty or invalid, indicating that the response did not contain the expected data, the method takes corrective action. It initiates a coroutine (StartCoroutine(FetchMessageContent())) to retry fetching message content after a short delay. This retry mechanism helps handle transient errors or delays in receiving valid data from the API.

If the response contains valid content, the text value of the first content item is deserialized into a Question object. Finally, the manager. The loadQuestion method is called to load and display the retrieved question within the application.

In conclusion, the integration of API requests within TrivAI plays a pivotal role in facilitating seamless communication between players and the AI. Through careful construction of HTTP requests, handling of response data, and error management, TrivAI ensures a robust and reliable interaction experience.

5.5. Package Manager

Within Unity, the Package Manager serves as a central hub for managing and integrating essential assets like BestHTTP and DoTween into TrivAI's development. This powerful tool streamlines the process of discovering, installing, updating, and removing packages, empowering developers to enhance their projects with additional functionalities, tools, and resources.

5.5.1. BestHTTP

Incorporating BestHTTP into TrivAI via the Package Manager is a pivotal step, elevating the project's networking capabilities to new heights. BestHTTP, a comprehensive HTTP library tailored for Unity, boasts an array of robust features and functionalities essential for handling diverse networking tasks with ease and efficiency. With support for various HTTP methods such as GET, POST, PUT, and DELETE, BestHTTP offers unparalleled versatility, making it an ideal choice for Unity projects, including TrivAI. Here's a closer look at some of the standout features of BestHTTP within the TrivAI ecosystem:

- Comprehensive HTTP Support: BestHTTP provides comprehensive support for various HTTP methods, enabling TrivAI to communicate with external APIs, servers, and web services.
- Flexible Request Configuration: With BestHTTP, TrivAI gains the flexibility to configure and customize HTTP requests according to specific requirements.
- Efficient Response Handling: BestHTTP simplifies the process of handling HTTP responses, offering intuitive interfaces for parsing, processing, and extracting relevant data.
- Robust Error Management: Exceptional error handling capabilities are integral to TrivAI's reliability and stability.

Examples of BestHTTP usage can be readily observed throughout the OpenAI chapter, where it facilitates various HTTP requests and interactions with the OpenAI API. From creating threads and posting messages to fetching content and executing thread runners, BestHTTP demonstrates its versatility and reliability in handling diverse networking tasks within TrivAI.

5.5.2. DOTween

DOTween is a powerful animation engine for Unity that simplifies the process of creating and managing tweens, easing functions, and animations. It provides a straightforward and intuitive API for animating properties such as position, rotation, scale, color, and more, allowing developers to create complex animations with ease.

One of the standout features of DOTween is its support for various easing functions, which enable developers to define the motion and acceleration of animations precisely. Whether it's a smooth linear movement, a gradual acceleration, and deceleration, or a custom easing curve, DOTween offers a wide range of options to achieve the desired animation effect.

For example, DOTween is used for the login background animation:

```
2 references
public void Initialize(string s)
{
    label?.SetText(s);
    label.font = fonts[Random.Range(0, fonts.Count)];
    label.alpha = Random.Range(minAlpha, maxAlpha);
    label.fontSize = Random.Range(minFontSize, maxFontSize);

    transform.rotation = Quaternion.Euler(0, 0, Random.Range(0, 360));
    transform.position = new Vector3(Random.Range(0, Screen.width), -300, 0);

    transform.DOMoveY(Screen.height + 300, Random.Range(minDuration, maxDuration)).SetEase(Ease.Linear).OnComplete(Reset);
}
```

In the provided code, DOTween is used to move background items smoothly across the screen with animations that have specified durations and easing functions.
The ‘transform.DOMoveY’ method is utilized to animate the Y position of the background items from their current positions to a target position, which is off-screen, over a specified duration. Additionally, the ‘OnComplete(Reset)’ method is used to trigger a callback function called Reset once the movement animation is complete. This function resets the properties of the background items, such as their position and text, to start a new animation cycle.

6. Future Improvements

This chapter explores potential directions for enhancing the TrivAI experience and engaging players in new and innovative ways.

1. User-Generated Content:

Players can contribute trivia questions and categories, fostering community engagement and expanding the game's content.

2. Multiplayer Mode:

Introducing real-time trivia battles encourages social interaction and competitive gaming among players.

3. Story Mode and Genre Expansion:

Incorporating RPG-inspired storylines and exploring diverse themes broadens TrivAI's appeal and offers deeper immersion.

4. Innovative Question Modes:

Experimenting with new question formats, like open-ended responses or "closest guess" challenges, adds variety and excitement. The future of TrivAI is filled with exciting possibilities for growth, innovation, and expansion. By embracing user-generated content, multiplayer modes, immersive storylines, and innovative question formats, TrivAI can continue to captivate and entertain players

7. Conclusion

In contemplating Socrates' wisdom, we're reminded of the perpetual pursuit of knowledge and understanding. While Socrates acknowledged human limitations in comprehension, AI appears to possess boundless knowledge. However, AI's grasp of understanding remains elusive, emphasizing the disparity between mere knowledge and true comprehension. This philosophical reflection resonates within TrivAI, where the quest for knowledge intersects with the pursuit of deeper understanding in the evolving realm of artificial intelligence and gaming.

In conclusion, the development path of TrivAI has been both challenging and rewarding. Throughout the process, the project encountered various obstacles, including technical complexities and time constraints. Despite these setbacks, the implementation of advanced technologies, such as OpenAI's GPT-4o and Unity's game engine platform, resulted in an overall notable game application.

TrivAI represents a significant endeavor in the realm of gaming applications, aiming to provide users with an engaging and intellectually stimulating experience. By addressing key challenges in game development such as replayability, maintenance costs, content creation, and platform compatibility, TrivAI demonstrates the potential of leveraging advanced technologies like OpenAI's GPT-4o and Unity game development platform. The integration of dynamic question generation, user-friendly interfaces, and innovative gameplay mechanics showcases the versatility and effectiveness of modern game development approaches. Moreover, the inclusion of features like help methods and a scoring/leveling system adds depth and engagement to the gaming experience. Overall, TrivAI stands as a testament to the power of innovation and technology in creating immersive and enjoyable gaming experiences with educational benefits.

In essence, TrivAI is more than just a project, it is a representation of my unwavering vision, creativity, and commitment.

References

[1]. Giattino, C., Mathieu, E., Broden, J., & Roser, M. (2023). Artificial Intelligence. Our World in Data. [Online] Available at: https://ourworldindata.org/artificial-intelligence

[2]. Tian, X. (2024). AI applications in video games and future expectations. Applied and Computational Engineering. [Online] Available at: https://www.researchgate.net/publication/379406019_AI_applications_in_video_games_and_future_expectations

[3]. Turing, A. (1950). Computing Machinery and Intelligence. Mind, 49, 433-460.

[4]. Srinivasan, A. (2022, January 31). The first of its kind AI Model- Samuel's Checkers Playing Program. Medium. [Online] Available at: https://medium.com/ibm-data-ai/the-first-of-its-kind-ai-model-samuels-checkers-playing-program-1b712fa4ab96

[5]. AI for game production. (2013, August 1). IEEE Conference Publication | IEEE Xplore. [Online] Available at: https://ieeexplore.ieee.org/document/6633663

[6]. Market.us. (2024, March 15). Generative AI in gaming market size | CAGR of 23.3%. [Online] Available at: https://market.us/report/generative-ai-in-gaming-market/

[7]. Jeong, H. J., Koo, E. Y., & Han, K. S. (2015). A study on the maintenance cost Estimation model for application Software by considering risks. Han'gug IT Seo'bi'seu Haghoeji, 14(3), 67–84. https://doi.org/10.9716/kits.2015.14.3.067

[8]. Riedl, M., & Zook, A. (n.d.). AI for Game Production. [Online] Available at: https://faculty.cc.gatech.edu/~riedl/pubs/cig13.pdf

[9]. IBM. (2023). IBM Watson. [Online] Available at: https://www.ibm.com/watson

[10]. Anyoha, R. (2017). The history of artificial intelligence. Science in the News. [Online] Available at: https://sitn.hms.harvard.edu/flash/2017/history-artificial-intelligence/

[11]. Giattino, C., Mathieu, E., Broden, J., & Roser, M. (2023). Artificial Intelligence. Our World in Data. [Online] Available at: https://ourworldindata.org/artificial-intelligence

[12]. gitnux.org. (n.d.). Must-Know Mobile Games Statistics [Latest Report] • Gitnux. [Online] Available at: https://gitnux.org/mobile-games-statistics/